

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



ĐỒ ÁN ĐA NGÀNH HƯỚNG TRÍ TUỆ NHÂN TẠO

NHÀ THÔNG MINH ỨNG DỤNG KẾT NỐI
VẠN VẬT

Giảng viên hướng dẫn: Thầy Lê Bình Đăng

Lớp: L03

Sinh viên thực hiện: Nguyễn Thiên Hào (**Nhóm trưởng**) — 2310853

Hà Thúc Thanh Phong — 2212549

Nguyễn Đức Toàn — 2313485

Thân Thiên Kim — 2311795

Phan Ngọc Phương Trinh — 2313582

THÀNH PHỐ HỒ CHÍ MINH – 2026



Danh sách thành viên

STT	Họ tên	MSSV	Phân công công việc
1	Nguyễn Thiên Hào (Nhóm trưởng)	2310853	- Thực hiện Backend - Báo cáo tiến độ và đánh giá hệ thống
2	Hà Thúc Thanh Phong	2212549	- Thực hiện UI Frontend - Viết API doc cho Web
3	Nguyễn Đức Toàn	2313485	- Thực hiện Backend và kết hợp học máy - Báo cáo tiến độ và đánh giá hệ thống
4	Thân Thiên Kim	2311795	- Thực hiện UI Frontend - Báo cáo tiến độ và viết báo cáo
5	Phan Ngọc Phương Trình	2313582	- Kết nối Yolo:Bit - Báo cáo tiến độ và đánh giá hệ thống

Mục lục

1	Giới Thiệu & Phân Tích Yêu Cầu	1
1.1	Giới thiệu	1
1.2	Phân tích yêu cầu	1
2	Công Nghệ Gateway và thiết bị Trong Dự Án Yolo:Home	3
2.1	Bộ Xử Lý Trung Tâm - Yolo:Bit	3
2.2	Môi trường lập trình - OhStem App	3
2.3	Hệ Thống Cảm Biến (Đầu Vào)	3
2.4	Hệ Thống Hiển Thị (Đầu Ra)	3
2.5	Hệ Thống Điều Khiển (Đầu Ra)	4
2.6	Hệ thống MQTT Server và AI tự xây dựng	4
2.6.1	Tìm hiểu công nghệ MQTT Broker - Eclipse Mosquitto	4
3	Công Nghệ Website Trong Dự Án Yolo:Home	6
3.1	Frontend	6
3.2	Backend	6
3.2.1	Công nghệ MongoDB	6
3.2.2	Công nghệ Decision Tree trong điều khiển rèm thông minh	8
4	Thiết kế	9
4.1	Thiết kế tổng thể	9
4.1.1	Kiến trúc tổng thể	9
4.1.2	Luồng hoạt động chính	11
4.2	Module 1 — Đo đạc và hiển thị môi trường	15
4.2.1	Mục tiêu	15
4.2.2	Thành phần liên quan	15
4.2.3	Luồng xử lý	15
4.2.4	Giao diện người dùng	16
4.2.5	Hạn chế và hướng phát triển	16
4.3	Module 2 — Tự động hóa và cảnh báo ngưỡng	16
4.3.1	Mục tiêu	16
4.3.2	Trạng thái cài đặt	16
4.3.3	Thành phần liên quan	16
4.3.4	Luồng xử lý	17
4.3.5	Giao diện người dùng	17
4.3.6	Hạn chế và hướng phát triển	18
4.4	Module 3 — Giao diện điều khiển thiết bị	18

4.4.1	Mục tiêu	18
4.4.2	Trạng thái cài đặt	18
4.4.3	Thành phần liên quan	18
4.4.4	Luồng xử lý	19
4.4.5	Giao diện người dùng	19
4.4.6	Hạn chế và hướng phát triển	19
4.5	Module 4 — Lưu trữ lịch sử và ghi sự kiện	20
4.5.1	Mục tiêu	20
4.5.2	Trạng thái cài đặt	20
4.5.3	Thành phần liên quan	20
4.5.4	Luồng xử lý	20
4.5.5	Giao diện người dùng	21
4.5.6	Hạn chế và hướng phát triển	21
4.6	Module 5 — Điều khiển bằng giọng nói và gợi ý thông minh	21
4.6.1	Mục tiêu	21
4.6.2	Thành phần liên quan	21
4.6.3	Luồng xử lý	22
4.6.4	Giao diện người dùng	22
4.6.5	Hạn chế và hướng phát triển	22
4.7	Thiết kế AI: Điều khiển bằng giọng nói	23
4.8	Thiết kế AI: Gợi ý thông minh dựa trên dữ liệu môi trường	23
4.8.1	Xây dựng tập dữ liệu (Dataset)	23
4.8.2	Cấu trúc mô hình	24
4.8.3	Đánh giá kết quả (Evaluation)	24
4.8.4	Hướng phát triển	25
4.9	Yolo:Bit	25
4.9.1	1. Giai đoạn khởi tạo (Initialization)	25
4.9.2	2. Luồng thu thập dữ liệu và Giao tiếp Serial (Main Loop)	25
4.9.3	3. Luồng xử lý sự kiện (Event Manager)	26
4.9.4	4. Luồng đồng bộ trạng thái định kỳ (Timer Event)	26
4.10	Các kênh giao tiếp	26
5	Giao diện Website dự án	30
6	Tổng kết dự án	32
6.1	Kết quả đạt được	32
6.2	Hướng phát triển tương lai	32

1 Giới Thiệu & Phân Tích Yêu Cầu

1.1 Giới thiệu

Trong các hộ gia đình hiện nay, việc theo dõi và kiểm soát các chỉ số an toàn môi trường sống như nhiệt độ, độ ẩm và cường độ ánh sáng đóng vai trò vô cùng quan trọng. Sự biến động bất thường của môi trường không chỉ ảnh hưởng trực tiếp đến sức khỏe của các thành viên trong gia đình mà còn tiềm ẩn nhiều nguy cơ cháy nổ nguy hiểm (khi nhiệt độ vượt ngưỡng quá cao), phát sinh ẩm mốc gây hại cho các thiết bị điện tử, hoặc không gian sống thiếu ánh sáng thích hợp. Do đó, nhu cầu giám sát an toàn môi trường sống luôn là ưu tiên hàng đầu.

Tuy nhiên, việc kiểm soát các thiết bị chấp hành (như quạt gió, đèn chiếu sáng, hay động cơ rèm cửa) một cách thủ công thường bộc lộ nhiều hạn chế lớn. Phương pháp vận hành thủ công đòi hỏi con người phải liên tục túc trực để nhận biết biến động và đưa ra phản ứng. Điều này gây tiêu tốn thời gian và hoàn toàn không thể bảo đảm phản ứng kịp thời khi có các sự cố bất ngờ xảy ra, đặc biệt là trong những khoảng thời gian người dùng vắng nhà hoặc đang ngủ sâu giấc.

Sự phát triển mạnh mẽ của công nghệ mạng không dây cùng trí tuệ nhân tạo đã mở ra xu thế tự động hóa và điều khiển thông minh trong các hệ sinh thái nhà thông minh (Smart Home). Bằng cách kết hợp linh hoạt giữa các cảm biến vật lý thu thập dữ liệu thời gian thực, giao thức giao tiếp bất đồng bộ hiệu năng cao như MQTT, và các mô hình học máy (nhận diện giọng nói tiếng Việt Vosk STT, phân loại ý định ML, hoặc cây quyết định Decision Tree tự động hóa), hệ thống có thể chủ động giám sát an toàn và vận hành tối ưu năng lượng, kiến tạo một không gian sống tiện nghi, tự động và an toàn tuyệt đối.

1.2 Phân tích yêu cầu

Dự án có các tính năng tự động hóa, như giám sát môi trường và điều khiển tự động các thiết bị, nhà thông minh hiện nay được bổ sung thêm công nghệ kết nối vạn vật, để nó có thể chia sẻ dữ liệu qua mạng Internet hay điều khiển thông minh với công nghệ trí tuệ nhân tạo.

Module 1 — Đo đạc & Hiển thị (Môi trường)

- **Mục đích:** Thu thập liên tục, xử lý và hiển thị trực quan các thông số môi trường (nhiệt độ, độ ẩm, cường độ ánh sáng) tại thời gian thực.
- **FR (Yêu cầu chức năng):**
 - Lấy mẫu định kỳ với tần suất có thể tùy chỉnh (ví dụ: 1-60 giây/lần).
 - Chuẩn hóa dữ liệu thô sang các đơn vị đo lường chuẩn ($^{\circ}\text{C}$, %RH, lux).
 - Hiển thị dữ liệu realtime trên Dashboard của App/Web dưới dạng số và biểu đồ cơ bản.

Module 2 — Tự động hóa ngưỡng & Cảnh báo

- **Mục đích:** Tự động hóa hoạt động của thiết bị dựa trên bộ luật (rules) điều kiện môi trường và kịp thời gửi cảnh báo khi có sự cố.

- **FR:**

- Cấu hình (CRUD) các bộ ngưỡng ngày/đêm và các kịch bản tự động.
- Trigger hành động khi vượt ngưỡng: Tự bật đèn khi thiếu sáng, bật quạt/điều hòa khi nhiệt độ cao.
- Cơ chế Override: Tạm dừng kịch bản tự động nếu người dùng can thiệp thủ công.
- Gửi thông báo trạng thái hoặc cảnh báo khẩn cấp qua Push Notification (FCM/APNs), Email hoặc SMS.

Module 3 — Giao diện điều khiển thiết bị

- **Mục đích:** Cung cấp phương thức để người dùng chủ động kiểm soát, can thiệp thủ công và quản lý tập trung các thiết bị trong nhà.

- **FR:**

- Bật/tắt nhanh thiết bị đơn lẻ hoặc theo nhóm (grouping).
- Thiết lập lịch hẹn giờ (Scheduling / Timer) cho thiết bị.
- Đồng bộ trạng thái hai chiều (Device ↔ Server ↔ App) để luôn hiển thị đúng trạng thái thật của phần cứng.

Module 4 — Lưu trữ lịch sử & Sự kiện

- **Mục đích:** Lưu vết (log) toàn bộ hoạt động của hệ thống phục vụ cho việc tra cứu, gỡ lỗi và phân tích dữ liệu dài hạn.

- **FR:**

- Lưu trữ dữ liệu dạng Time-series cho sensor readings.
- Ghi nhận lịch sử trạng thái thiết bị và event logs (ai đã bật/tắt cái gì, vào lúc nào).
- Ghi log các sự kiện hệ thống: Cảnh báo tự động, lỗi kết nối thiết bị.
- Trích xuất dữ liệu lịch sử theo khoảng thời gian tùy chọn.

Module 5 — Điều khiển giọng nói & Gợi ý thông minh

- **Mục đích:** Nâng cao trải nghiệm tiện ích rảnh tay (hands-free) và cung cấp các đề xuất tự động giúp tối ưu hóa môi trường sống.

- **FR:**

- Chuyển đổi giọng nói thành văn bản (STT) và trích xuất ý định (Intent parsing) để ra lệnh cho thiết bị.
- Gợi ý ngữ cảnh thông minh: Khuyến người dùng mở rèm/cửa sổ khi thời tiết bên ngoài mát mẻ, hoặc đóng lại khi nắng gắt.

2 Công Nghệ Gateway và thiết bị Trong Dự Án Yolo:Home

2.1 Bộ Xử Lý Trung Tâm - Yolo:Bit

- **Vai trò:** Bộ xử lý trung tâm, nơi xử lý tín hiệu từ các cảm biến và điều khiển thiết bị.
- **Thông số:** Chip ESP32, hỗ trợ WiFi và Bluetooth, điện áp 3.3V.
- **Tích hợp:** Màn hình LED 5x5, nút nhấn A/B, cảm biến nhiệt độ/ánh sáng cơ bản.

2.2 Môi trường lập trình - OhStem App

OhStem App là nền tảng lập trình trực tuyến được sử dụng trong dự án. Các tính năng chính bao gồm lập trình kéo thả trực tuyến dựa trên ngôn ngữ Blockly của Google, hỗ trợ đa thiết bị như máy tính, điện thoại thông minh và máy tính bảng. App cho phép kết nối với Yolo:Bit qua USB (sử dụng driver CH340) hoặc Bluetooth, đồng thời hỗ trợ xuất, nhập dự án dưới dạng file .json và chia sẻ project qua đường dẫn. Người dùng có thể chạy thử chương trình hoặc nạp trực tiếp vào bộ nhớ của thiết bị. Ngoài ra, OhStem App còn tích hợp quy trình khôi phục cài đặt gốc bằng tổ hợp phím A + Reset.

Lưu ý: Trong hệ thống của nhóm, OhStem App **chỉ được sử dụng ở mức độ lập trình cho kit Yolo:Bit**. Các tính năng MQTT server (IoT) và Trí tuệ nhân tạo (AI) có sẵn trên OhStem **không được sử dụng**. Toàn bộ phần MQTT server và AI models được nhóm tự xây dựng riêng cho hệ thống.

2.3 Hệ Thống Cảm Biến (Đầu Vào)

a. Cảm biến nhiệt độ & độ ẩm DHT20

- **Chức năng:** Đo nhiệt độ và độ ẩm không khí.
- **Giao tiếp:** I2C. Kết nối: **P19 (SCL) - P20 (SDA)**.

b. Cảm biến ánh sáng

- **Chức năng:** Đo cường độ ánh sáng môi trường.
- **Nguyên lý:** Quang trở (trở kháng thay đổi theo ánh sáng), tín hiệu analog.
- **Kết nối:** Chân **P2** (ADC).

2.4 Hệ Thống Hiển Thị (Đầu Ra)

Màn hình LCD 16x2

- **Chức năng:** Hiển thị thông số cảm biến và thời gian tại chỗ.
- **Giao tiếp:** I2C. Kết nối: **P21 (SCL) - P23 (SDA)**.

2.5 Hệ Thống Điều Khiển (Đầu Ra)

a. Đèn LED RGB

- **Nguyên lý:** NeoPixel (WS2812), điều khiển từng đèn với màu RGB.
- **Kết nối:** Chân P1 (PWM).

b. Động cơ - Quạt mini

- **Nguyên lý:** Động cơ DC, điều khiển tốc độ bằng PWM.
- **Kết nối:** Chân P0.

c. Động cơ RC Servo

- **Chức năng:** Điều khiển góc quay (0-180°), mô phỏng đóng/mở cửa.
- **Kết nối:** Chân P4.

Bảng tổng kết kết nối các thiết bị

Thiết bị	Giao tiếp	Chân kết nối
Cảm biến DHT20	I2C	P19 (SCL) - P20 (SDA)
Màn hình LCD 16x2	I2C	P21 (SCL) - P23 (SDA)
Đèn LED RGB	Digital (PWM)	P1
Cảm biến ánh sáng	Analog (ADC)	P2
Quạt mini (Động cơ DC)	Digital (PWM)	P0
Động cơ RC Servo	Digital (PWM)	P4

Bảng 1: Bảng kết nối chi tiết các thiết bị ngoại vi với mạch Yolo:Bit

2.6 Hệ thống MQTT Server và AI tự xây dựng

Các thành phần IoT và AI trong dự án được nhóm tự xây dựng hoàn toàn. Cụ thể, nhóm tự triển khai máy chủ MQTT riêng để quản lý các kênh dữ liệu phục vụ cho việc gửi và nhận thông tin giữa Yolo:Bit và bảng điều khiển IoT. Bên cạnh đó, các mô hình trí tuệ nhân tạo như nhận diện giọng nói (Speech-to-Text) cũng được nhóm tự xây dựng và huấn luyện, hoàn toàn không sử dụng các dịch vụ AI có sẵn.

2.6.1 Tìm hiểu công nghệ MQTT Broker - Eclipse Mosquitto

Để chủ động trong việc trao đổi dữ liệu MQTT giữa các thiết bị trong hệ thống IoT, nhóm đã quyết định tự xây dựng máy chủ MQTT riêng bằng giải pháp mã nguồn mở **Eclipse Mosquitto**.

a. Gi thiệu về Eclipse Mosquitto

Eclipse Mosquitto là một message broker mã nguồn mở triển khai giao thức MQTT phiên bản 5.0, 3.1.1 và 3.1. Được viết bằng ngôn ngữ C/C++, Mosquitto hướng đến sự đơn giản, nhẹ nhàng và hiệu quả. Broker đóng

vai trò trung gian giữa các MQTT client, nhận tin nhắn từ publisher và chuyển tiếp đến subscriber phù hợp dựa trên topic đã đăng ký .

b. Kiến trúc kỹ thuật

Mosquitto sử dụng **kiến trúc single-thread, event-driven** . Toàn bộ kết nối và xử lý tin nhắn được thực hiện trên một luồng đơn, giúp tránh chi phí đồng bộ hóa và tiết kiệm tài nguyên. Cụ thể, Mosquitto chỉ tiêu tốn khoảng **60kB bộ nhớ chương trình** và **2-4kB heap cho mỗi client** .

Về cơ chế hoạt động, Mosquitto tuân theo mô hình publish/subscribe đặc trưng của MQTT: publisher gửi tin nhắn kèm topic, subscriber đăng ký nhận topic, broker chuyển tiếp tin nhắn đến đúng subscriber .

c. Tính năng bảo mật và tiện ích

Mosquitto hỗ trợ đầy đủ các cơ chế bảo mật cần thiết: xác thực username/password, mã hóa TLS/SSL, ACL phân quyền theo topic, và Dynamic Security Plugin cho phép quản lý theo thời gian thực .

Ngoài ra, Mosquitto còn hỗ trợ:

- **WebSocket:** Cho phép browser kết nối trực tiếp đến MQTT broker mà không cần backend trung gian , rất hữu ích cho bảng điều khiển IoT real-time.
- **Công cụ dòng lệnh:** Các tiện ích `mosquitto_pub`, `mosquitto_sub`, `mosquitto_passwd` giúp kiểm tra và gỡ lỗi dễ dàng .

Ví dụ sử dụng:

```
mosquitto_pub -t "home/livingroom/device/led/set" -m '{"action":"on"}'  
mosquitto_sub -t "home/livingroom/device/+state" -v
```

d. Đánh giá ưu điểm

Mosquitto có các ưu điểm nổi bật sau:

- **Nhẹ và hiệu quả:** Yêu cầu tài nguyên tối thiểu, phù hợp chạy trên máy chủ cấu hình thấp hoặc thiết bị nhúng.
- **Độ trễ thấp:** Đạt độ trễ dưới 1 mili giây trong điều kiện tải nhẹ .
- **Ổn định:** Khả năng phục hồi nhanh sau sự cố .
- **Đa nền tảng:** Chạy được trên Linux, Windows, macOS, Docker và cả ESP32 .
- **Dễ cài đặt:** Chỉ một dòng lệnh `sudo apt-get install mosquitto` trên Linux .
- **Miễn phí và mã nguồn mở:** Được hỗ trợ bởi Eclipse Foundation với cộng đồng lớn .

e. Lý do lựa chọn Mosquitto cho dự án

Nhóm quyết định lựa chọn Mosquitto làm MQTT broker dựa trên các lý do sau:

1. **Phù hợp với kiến trúc topic của dự án:** Hỗ trợ wildcard '+' và '#', đáp ứng cấu trúc topic chi tiết như 'home/livingroom/device/+/state'.
2. **Chủ động hoàn toàn về hạ tầng:** Không phụ thuộc nhà cung cấp dịch vụ, dữ liệu do nhóm tự quản lý.
3. **Không bị giới hạn (Limit):** Tránh các giới hạn của server miễn phí (số kết nối, số tin nhắn/ngày).
4. **Hiệu năng trong môi trường LAN:** Đặt broker cùng mạng nội bộ với Yolo:Bit, đảm bảo thời gian thực.
5. **Dễ kiểm thử:** Công cụ dòng lệnh giúp kiểm thử nhanh luồng dữ liệu theo tài liệu API.

3 Công Nghệ Website Trong Dự Án Yolo:Home

3.1 Frontend

Để xây dựng giao diện người dùng hiện đại, trực quan và tối ưu hóa trải nghiệm điều khiển thời gian thực, dự án YOLOHome sử dụng các công nghệ Frontend cốt lõi sau:

- **React 19:** Thư viện JavaScript mã nguồn mở hàng đầu để xây dựng giao diện người dùng dạng Single Page Application (SPA) thông qua kiến trúc hướng thành phần (Component-driven design).
- **Vite:** Công cụ build và bundler thế hệ mới với hiệu năng cực cao, sử dụng ES Modules bản địa giúp quá trình HMR (Hot Module Replacement) diễn ra tức thời khi phát triển và tối ưu hóa kích thước bundle khi đóng gói.
- **Tailwind CSS:** Framework CSS tiện ích (utility-first) giúp thiết kế giao diện nhanh chóng, đảm bảo tính nhất quán của hệ thống thiết kế và khả năng hiển thị tương thích hoàn hảo trên nhiều kích thước màn hình (Responsive Design).
- **RecordRTC:** Thư viện hỗ trợ ghi âm chất lượng cao trên trình duyệt Web, cho phép thu nhận giọng nói mono WAV với tần số lấy mẫu 16kHz chuẩn xác phục vụ dịch vụ nhận diện tiếng Việt.

3.2 Backend

3.2.1 Công nghệ MongoDB

MongoDB là một hệ quản trị cơ sở dữ liệu NoSQL, được thiết kế để lưu trữ dữ liệu dưới dạng tài liệu (document-oriented). Khác với các hệ quản trị cơ sở dữ liệu quan hệ truyền thống, MongoDB không sử dụng bảng (table) mà sử dụng các collection và document. Dữ liệu trong MongoDB được lưu dưới dạng BSON (Binary JSON), giúp xử lý nhanh và linh hoạt hơn.

Mô hình dữ liệu Trong MongoDB, dữ liệu được tổ chức theo ba cấp:

- **Document:** Đơn vị dữ liệu cơ bản, có cấu trúc dạng JSON.
- **Collection:** Tập hợp các document, tương tự như bảng trong SQL.
- **Database:** Chứa nhiều collection.

Các tính năng chính MongoDB cung cấp nhiều tính năng mạnh mẽ:

- **Schema linh hoạt:** Không cần định nghĩa cấu trúc dữ liệu trước, cho phép lưu trữ dữ liệu đa dạng.
- **Hiệu năng cao:** Tối ưu cho các thao tác đọc và ghi dữ liệu lớn.
- **Indexing:** Hỗ trợ nhiều loại chỉ mục giúp tăng tốc truy vấn.
- **Aggregation:** Cho phép xử lý và phân tích dữ liệu thông qua pipeline.
- **Replication:** Sao chép dữ liệu giữa nhiều server để đảm bảo tính sẵn sàng cao.
- **Sharding:** Phân tán dữ liệu trên nhiều máy chủ để mở rộng hệ thống.
- **Transaction:** Hỗ trợ giao dịch đảm bảo tính toàn vẹn dữ liệu.

Ưu điểm và nhược điểm **Ưu điểm:**

- Linh hoạt trong thiết kế dữ liệu
- Dễ mở rộng và phù hợp với hệ thống lớn
- Tốc độ xử lý nhanh

Nhược điểm:

- Không phù hợp với hệ thống yêu cầu quan hệ dữ liệu phức tạp
- Transaction không mạnh bằng các hệ quản trị SQL truyền thống

Ứng dụng MongoDB được sử dụng rộng rãi trong:

- Ứng dụng web hiện đại (Node.js, React)
- Hệ thống IoT
- Phân tích dữ liệu lớn (Big Data)
- Trí tuệ nhân tạo (AI)

Kết luận MongoDB là một giải pháp cơ sở dữ liệu hiện đại, phù hợp với các hệ thống cần xử lý dữ liệu linh hoạt và có khả năng mở rộng cao. Tuy nhiên, việc lựa chọn sử dụng MongoDB hay cơ sở dữ liệu quan hệ cần dựa trên yêu cầu cụ thể của hệ thống.

3.2.2 Công nghệ Decision Tree trong điều khiển rềm thông minh

Hệ thống điều khiển rềm thông minh được xây dựng dựa trên thuật học có giám sát (supervised learning), cụ thể là **Decision Tree Classifier** từ thư viện **scikit-learn** của Python.

a. Giải thuật Decision Tree

Decision Tree (cây quyết định) là một phương pháp học phi tham số (non-parametric), hoạt động dựa trên chiến lược **chia để trị (divide and conquer)**. Cấu trúc của cây bao gồm:

- **Nút gốc (Root Node):** Điểm bắt đầu, chứa toàn bộ dữ liệu huấn luyện.
- **Nút quyết định (Decision Nodes):** Các nút bên trong, thực hiện việc phân chia dữ liệu dựa trên một đặc trưng và ngưỡng cụ thể. Mỗi nút đặt ra một câu hỏi có/không về giá trị đầu vào.
- **Nút lá (Leaf Nodes):** Các nút cuối cùng, đưa ra kết quả phân loại (quyết định cuối cùng).

Thuật toán xây dựng cây bằng cách tìm kiếm đặc trưng và ngưỡng phân chia tốt nhất tại mỗi nút, sao cho các nhánh con có độ thuần khiết (purity) cao nhất. Chất lượng của phép chia được đo bằng các chỉ số như **Gini impurity** (độ bất tinh Gini) hoặc **Entropy** (đo độ hỗn loạn của dữ liệu). Quá trình này lặp lại cho đến khi đạt điều kiện dừng, chẳng hạn như đạt độ sâu tối đa hoặc số lượng mẫu tại nút lá đạt ngưỡng tối thiểu.

b. Công cụ hỗ trợ - Scikit-learn

Để hiện thực Decision Tree, hệ thống sử dụng lớp **DecisionTreeClassifier** từ thư viện **scikit-learn**. Thư viện này cung cấp các tham số quan trọng để điều chỉnh mô hình:

- **criterion:** Hàm đánh giá chất lượng phân chia, có thể chọn "**gini**" (độ bất tinh Gini) hoặc "**entropy**" (lượng thông tin).
- **max_depth:** Độ sâu tối đa của cây, giúp kiểm soát hiện tượng overfitting.
- **min_samples_split:** Số mẫu tối thiểu để phân chia một nút nội bộ.
- **min_samples_leaf:** Số mẫu tối thiểu tại một nút lá, giúp làm mịn mô hình.
- **random_state:** Kiểm soát tính ngẫu nhiên để đảm bảo khả năng tái lập kết quả.

Các tham số này được tinh chỉnh để đạt được mô hình có độ chính xác cao và tránh overfitting.

c. Đánh giá ưu điểm và hạn chế

Decision Tree có nhiều ưu điểm nổi bật:

- **Dễ hiểu và giải thích được (Interpretable):** Cấu trúc cây trực quan, có thể biểu diễn dưới dạng các luật "if-then-else", giúp dễ dàng kiểm tra và lý giải quyết định của mô hình.

- **Yêu cầu tiền xử lý dữ liệu tối thiểu:** Không cần chuẩn hóa hay tạo biến giả (dummy variables).
- **Chi phí dự đoán thấp:** Thời gian dự đoán tỷ lệ thuận với độ sâu của cây (logarithmic), rất phù hợp với các ứng dụng thời gian thực.
- **Xử lý được cả dữ liệu số và phân loại:** Linh hoạt với nhiều loại dữ liệu đầu vào.
- **Cung cấp độ quan trọng của đặc trưng (feature importance):** Cho biết đặc trưng nào đóng góp nhiều nhất vào quyết định.

Bên cạnh đó, Decision Tree cũng có một số hạn chế cần lưu ý:

- **Dễ bị overfitting:** Cây quá phức tạp có thể học luôn cả nhiễu trong dữ liệu. Khắc phục bằng các tham số như `max_depth` hoặc `min_samples_leaf`.
- **Không ổn định:** Những thay đổi nhỏ trong dữ liệu có thể tạo ra cây hoàn toàn khác.
- **Không nắm bắt được các mẫu phức tạp:** So với các mô hình ensemble như Random Forest hay Gradient Boosting, cây đơn lẻ có sức biểu diễn hạn chế hơn.

d. Ứng dụng trong hệ thống điều khiển rèm

Trong hệ thống này, Decision Tree được huấn luyện để học các luật điều khiển rèm từ dữ liệu mẫu. Cụ thể, ba tín hiệu cảm biến đầu vào gồm ánh sáng (0-100%), nhiệt độ (-10°C đến 50°C) và độ ẩm (0-100%) được sử dụng để huấn luyện mô hình. Nhân đầu ra là hành động đóng (0) hoặc mở (1) rèm.

Mô hình được huấn luyện trên 10.000 mẫu dữ liệu nhân tạo, sinh ra từ các luật điều khiển do chuyên gia định nghĩa (ví dụ: ánh sáng quá mạnh thì đóng rèm, nhiệt độ thấp kết hợp ánh sáng cao thì mở rèm). Sau khi huấn luyện, Decision Tree có thể dự đoán nhanh chóng hành động phù hợp cho bất kỳ bộ giá trị cảm biến nào. Mô hình sau đó được lưu lại (bằng pickle) và triển khai trong backend server, sẵn sàng nhận dữ liệu cảm biến thực tế để đưa ra quyết định đóng/mở rèm một cách tự động và thông minh.

4 Thiết kế

4.1 Thiết kế tổng thể

4.1.1 Kiến trúc tổng thể

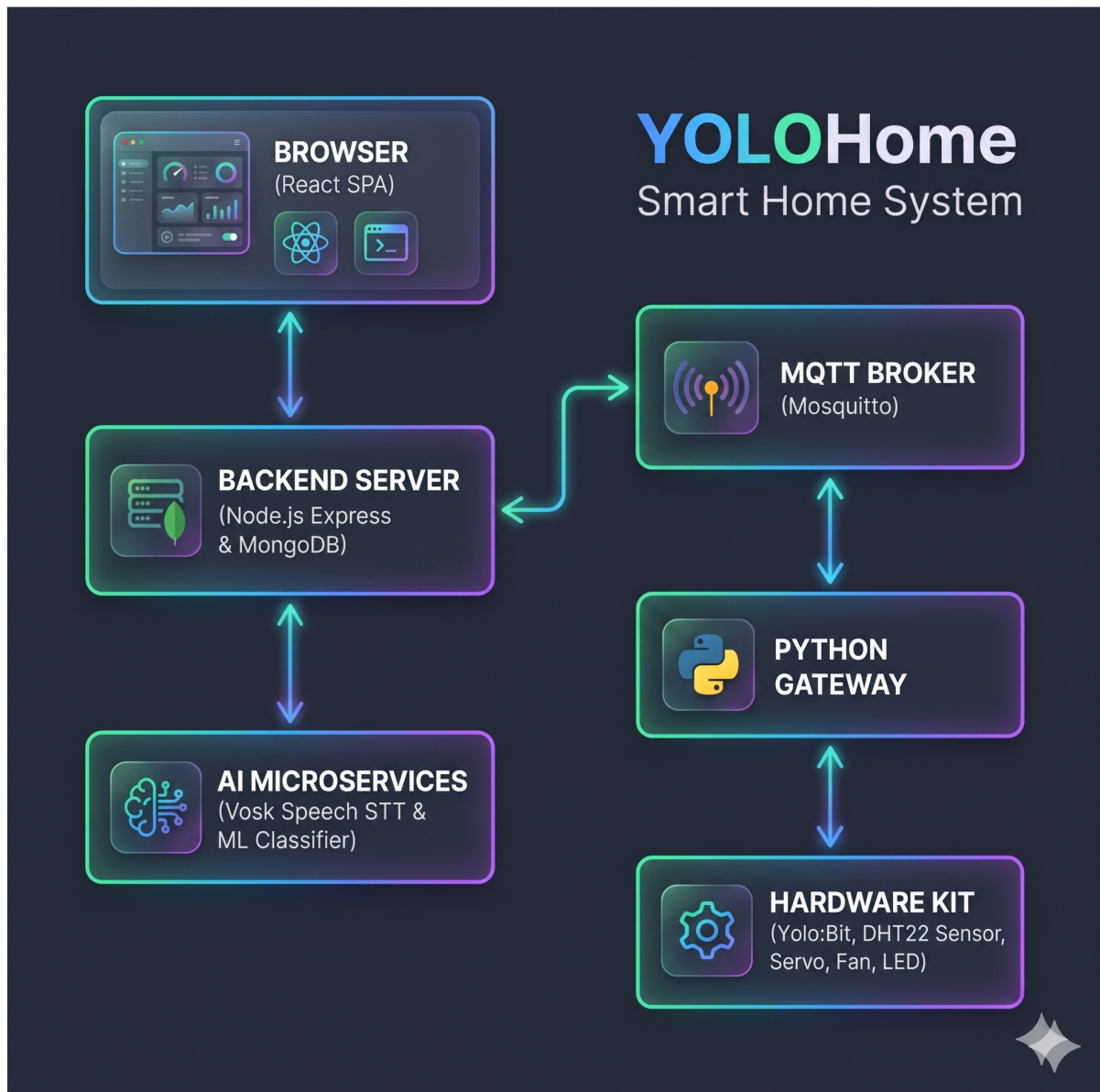
Hệ thống YOLOHome được thiết kế và xây dựng như một giải pháp giám sát và điều khiển nhà thông minh hoàn chỉnh, kết hợp hài hòa giữa các tầng công nghệ từ phần cứng đến phần mềm. Kiến trúc hệ thống bao gồm bốn thành phần cốt lõi:

- **Giao diện người dùng Web SPA:** Được phát triển trên nền tảng thư viện React 19 kết hợp công cụ build Vite, chạy trên môi trường containerized phục vụ bởi Nginx. Giao diện cung cấp khả năng quan sát

trực quan các thông số môi trường theo thời gian thực và cho phép bật/tắt thiết bị thủ công cùng tính năng Push-to-Talk điều khiển bằng giọng nói.

- **Tầng Backend REST API & MQTT Layer:** Sử dụng framework Express 5 chạy trên nền tảng Node.js kết hợp với hệ quản trị cơ sở dữ liệu phi quan hệ MongoDB. Tầng này chịu trách nhiệm điều phối các yêu cầu REST từ client, tích hợp các bộ chuyển dịch AI, lưu trữ dữ liệu snapshot môi trường, vết điều khiển (ControlTrace), nhật ký cảnh báo (Alert), và xuất bản các lệnh tới các chủ đề (topics) trên MQTT Broker Mosquitto.
- **Cầu nối trung gian Gateway:** Viết bằng ngôn ngữ lập trình Python, đóng vai trò chuyển đổi giao thức hai chiều giữa mạng tin nhắn MQTT phi tập trung và Kit phần cứng Yolo:Bit kết nối vật lý qua đường truyền USB Serial với băng thông ổn định. Gateway chịu trách nhiệm bóc tách các gói tin Serial dạng rút gọn và thực thi logic tự động hóa cục bộ dựa trên ngưỡng cấu hình hoặc mô hình AI Decision Tree.
- **Kit phần cứng vật lý:** Sử dụng vi điều khiển Yolo:Bit kết hợp hệ cảm biến đo đạc môi trường DHT22 (đo nhiệt độ, độ ẩm), quang trở LDR/BH1750 (đo ánh sáng) cùng các thiết bị chấp hành thực tế bao gồm đèn LED chiếu sáng, Quạt thông gió động cơ DC (Fan) và Động cơ Servo điều khiển rèm cửa.
- **Các dịch vụ phụ trợ AI:** Bao gồm dịch vụ nhận dạng giọng nói tiếng Việt Vosk STT và mô hình phân loại ý định ML được triển khai dưới dạng microservices độc lập, giao tiếp qua REST API với tầng Backend để cung cấp khả năng điều khiển bằng giọng nói thông minh.

Dưới đây là sơ đồ kiến trúc tổng thể của hệ thống. Trong quá trình phát triển, sơ đồ này được thiết kế và mô tả bằng ngôn ngữ Mermaid để phục vụ cho việc tự động kết xuất hình ảnh.



Hình 1: Sơ đồ kiến trúc tổng thể hệ thống YOLOHome

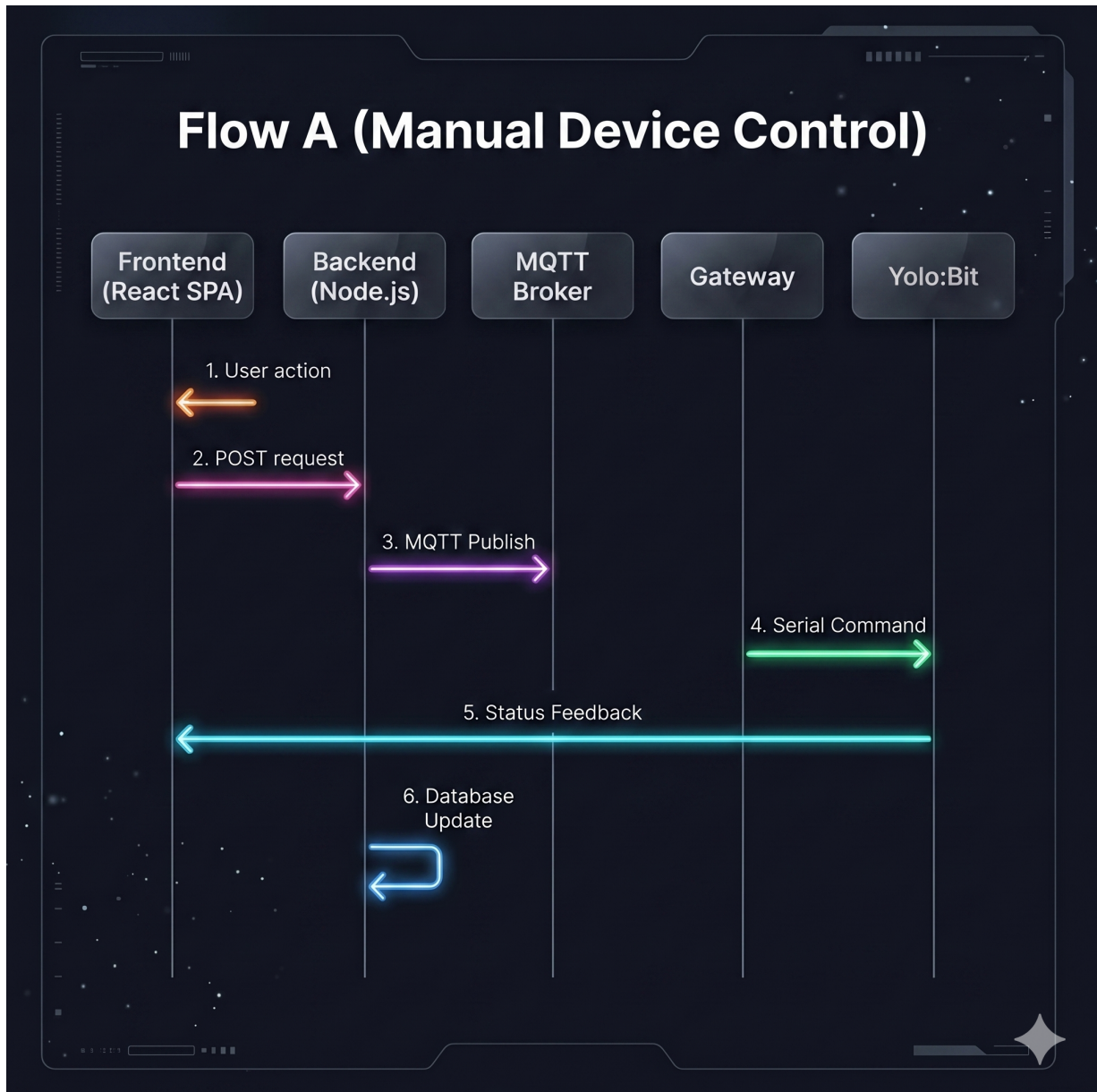
Sự tương tác giữa các lớp trong Figure 1 diễn ra theo mô hình hỗn hợp: Trình duyệt giao tiếp đồng bộ qua REST HTTP với Backend Express, trong khi Backend, Gateway và Kit phần cứng giao tiếp bất đồng bộ thông qua MQTT Broker Mosquitto và giao tiếp điểm-điểm qua Serial UART.

4.1.2 Luồng hoạt động chính

Hệ thống vận hành dựa trên ba luồng hoạt động cốt lõi tương ứng với các kịch bản sử dụng thực tế bao gồm điều khiển thủ công, tự động hóa theo ngưỡng an toàn môi trường và điều khiển bằng giọng nói.

Luồng hoạt động A: Điều khiển thiết bị thủ công

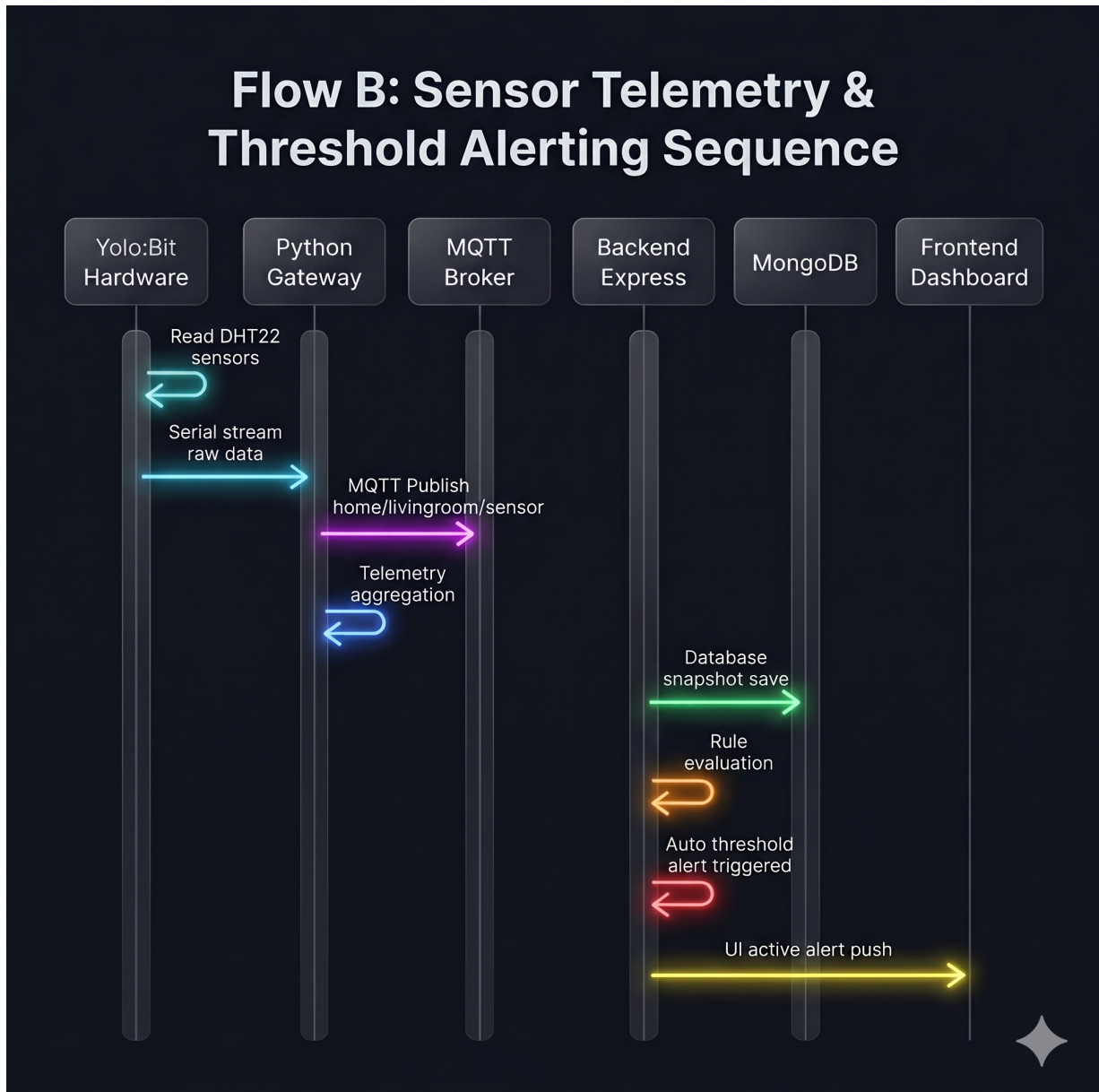
Khi người dùng thực hiện bật/tắt thiết bị trên giao diện quản lý thiết bị, yêu cầu HTTP POST đồng bộ được gửi tới Backend. Backend kiểm tra quyền và ghi log trước khi chuyển đổi lệnh thành bản tin MQTT truyền xuống Gateway. Gateway nhận bản tin thông qua MQTT, thực hiện giới hạn tần suất lệnh để bảo vệ phần cứng, mã hóa lệnh thành định dạng Serial rút gọn gửi qua cổng UART vật lý của Yolo:Bit. Yolo:Bit sau khi tác động Relay vật lý sẽ phản hồi lại khung xác nhận để Gateway cập nhật trạng thái lên Broker giúp Backend đồng bộ vào MongoDB.



Hình 2: Lược đồ tuần tự luồng điều khiển thiết bị thủ công (Flow A)

Luồng hoạt động B: Thu thập cảm biến và Cảnh báo ngưỡng

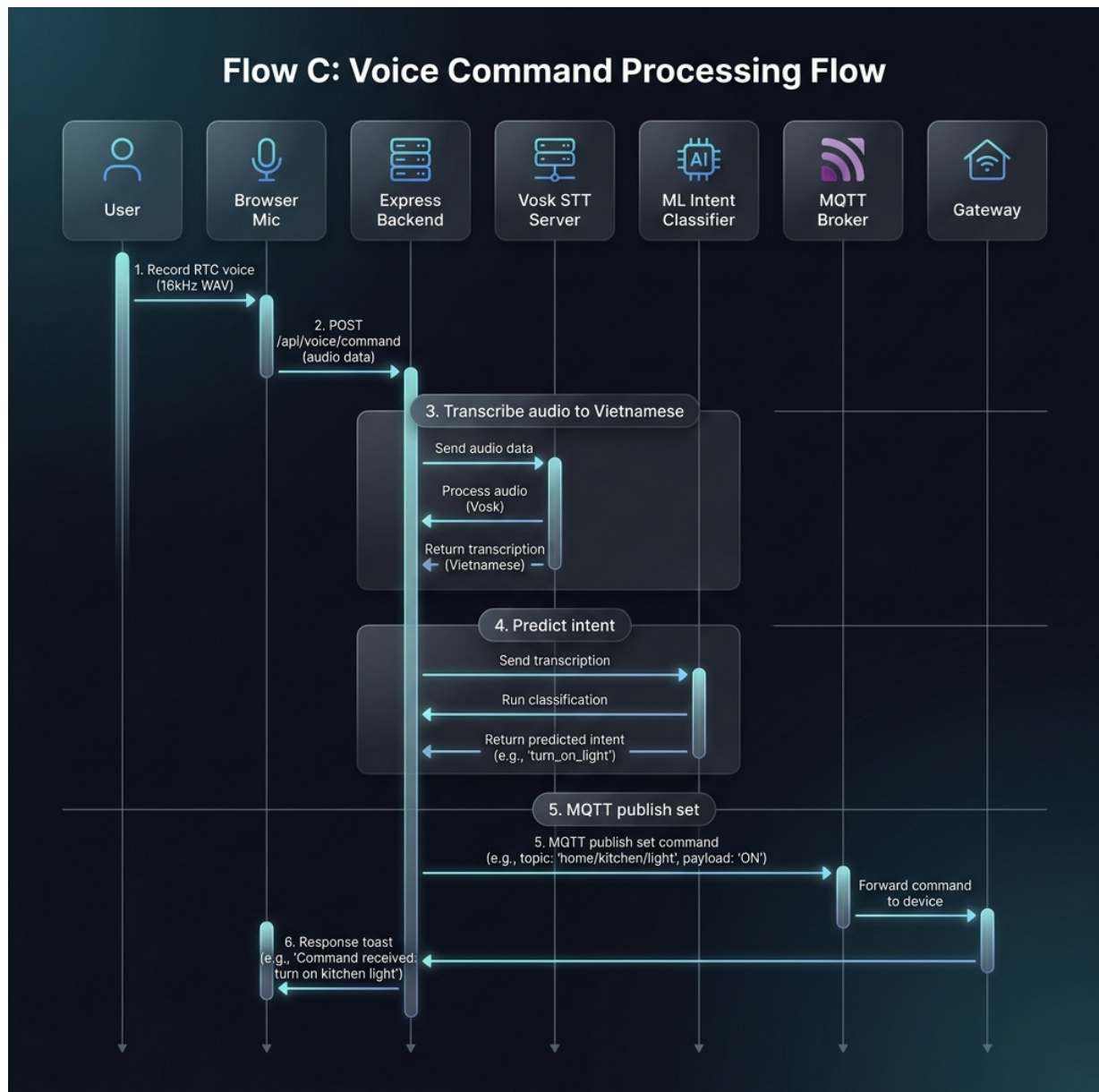
Kit Yolo:Bit đo đạc liên tục các thông số vật lý của môi trường thông qua các cảm biến nhiệt độ, độ ẩm và ánh sáng. Định kỳ, các thông số này được đóng gói và truyền lên Gateway Python dưới dạng các khung Serial nhỏ. Gateway bóc tách, biên dịch và đăng tải (publish) các thông số lên các topic MQTT tương ứng. Backend Express lắng nghe, tích lũy các chỉ số vào bộ đệm snapshot. Khi nhận đủ 3 chỉ số, Backend tiến hành ghi snapshot cảm biến vào MongoDB và kiểm tra tính an toàn dựa trên tập cấu hình ngưỡng để chủ động sinh cảnh báo và tự động kích hoạt thiết bị phần cứng để khắc phục sự cố.



Hình 3: *Lược đồ tuần tự luồng thu thập cảm biến và xử lý cảnh báo ngưỡng (Flow B)*

Luồng hoạt động C: Điều khiển bằng giọng nói

Khi người dùng tương tác bằng giọng nói trên giao diện Web (Push-to-Talk), trình duyệt sử dụng thư viện RecordRTC ghi âm luồng tiếng nói định dạng mono WAV tần số 16kHz gửi POST lên Backend. Backend Express tiếp nhận file âm thanh, chuyển tiếp qua luồng nhị phân tới dịch vụ Vosk STT (cổng 8500) để nhận dạng văn bản tiếng Việt. Chuỗi văn bản sau đó được gửi tiếp tới dịch vụ ML (cổng 8000) để phân loại ý định thành định dạng lệnh chuẩn. Backend nhận kết quả phân tích, tiến hành chuẩn hóa lệnh rồi xuất bản bản tin MQTT xuống Gateway để điều khiển thiết bị vật lý, đồng thời phản hồi kết quả trực quan dạng Toast trên màn hình Web.



Hình 4: Lược đồ tuần tự luồng điều khiển bằng giọng nói tiếng Việt (Flow C)

4.2 Module 1 — Đo đạc và hiển thị môi trường

4.2.1 Mục tiêu

Do lường các chỉ số môi trường sống thực tế bao gồm nhiệt độ, độ ẩm và cường độ ánh sáng trong nhà, sau đó gửi dữ liệu thu thập lên hệ thống và hiển thị trực quan các giá trị này lên giao diện Web Dashboard của người dùng theo thời gian thực để hỗ trợ giám sát an toàn.

4.2.2 Thành phần liên quan

Các thành phần và tệp tin mã nguồn tham gia trực tiếp vào module bao gồm:

- **Giao diện Frontend:** `Dashboard.js` (dòng 6–10, 37–59, 75, 128–149) chịu trách nhiệm tạo trạng thái lưu dữ liệu cảm biến, xây dựng hàm gọi API và thiết lập chu kỳ polling tự động.
- **Dịch vụ Backend Node.js:** `sensor.js` (router GET `/api/sensors/latest`), `sensorController.js` (dòng 5–15), `sensorService.js` (dòng 5–8) và `mqttService.js` (dòng 152–199) để xử lý định tuyến, truy vấn cơ sở dữ liệu MongoDB và gom dữ liệu cảm biến MQTT.
- **Dịch vụ Python Gateway:** `run.py` (dòng 244–246) để chạy luồng đọc Serial nền, `controller.py` (dòng 291–333) để phân tích khung truyền Serial và `default_adapter.py` (dòng 53–73) để biên dịch ánh xạ cổng cảm biến.
- **Cấu hình tệp tin:** `config.yml` (dòng 75–78) cấu hình các ký tự viết tắt tương ứng với tên cổng cảm biến.

4.2.3 Luồng xử lý

Luồng thu thập và hiển thị cảm biến môi trường được xử lý qua 8 bước cụ thể như mô tả trong lược đồ tuần tự của Figure 3:

- Hệ thống phần cứng Kit Yolo:Bit đo đạc các giá trị vật lý và định kỳ truyền lên Gateway Python qua kết nối cáp USB Serial dạng chuỗi ký tự viết tắt liên tục, ví dụ: `!T:27.5#!H:60#!Lu:350#`.
- Luồng đọc nền Serial của Gateway Python (`run.py`) lắng nghe cổng COM vật lý và chuyển chuỗi nhận được cho hàm `MainController._on_serial`.
- Bộ phân tích cú pháp của Gateway sử dụng biểu thức chính quy tách chuỗi thành các khung truyền con có dạng cấu trúc `!<ABBR>:<VALUE>#`.
- Hàm chuyển đổi `DefaultDataAdapter.from_serial` biên dịch các mã viết tắt cổng (T, H, Lu) thành tên cảm biến tiếng Anh tương thích (`temperature`, `humidity`, `light`) dựa trên cấu hình trong `config.yml`.
- Gateway thực hiện xuất bản (publish) dữ liệu JSON thô lên Broker MQTT Mosquitto với cấu trúc topic chuẩn, ví dụ: `home/livingroom/sensor/temperature` chứa payload `{"value": "27.5"}`.

- Backend Node.js nhận được tin nhắn qua dịch vụ `mqttService.js`, trích xuất giá trị số và đưa vào bộ đệm gom snapshot tạm thời có tên `sensorSnapshotBuffer`.
- Khi bộ đệm gom đầy đủ cả 3 thông số nhiệt độ, độ ẩm và cường độ ánh sáng, Backend gọi `SensorService.saveSensorSnapshot` ghi dữ liệu snapshot môi trường hoàn chỉnh vào MongoDB và xóa sạch bộ đệm để bắt đầu chu kỳ tiếp theo.
- Trình duyệt của người dùng tại giao diện Dashboard liên tục gọi HTTP GET đồng bộ tới endpoint `/api/sensors/latest` mỗi 5 giây thông qua hàm `setInterval`. Dữ liệu snapshot mới nhất nhận về được cập nhật lên state và tự động vẽ lại các thể hiển thị trên giao diện.

4.2.4 Giao diện người dùng

Trạng thái hiển thị dữ liệu cảm biến được tích hợp trực quan trên trang Dashboard chính (`/dashboard`). Giao diện hiển thị bao gồm lưới 3 thẻ (cards) riêng biệt: thẻ Nhiệt độ (hiển thị đơn vị °C), thẻ Độ ẩm (hiển thị đơn vị %) và thẻ Cường độ ánh sáng (hiển thị đơn vị lx) như mô tả tại nhân hình ảnh Figure 5.

4.2.5 Hạn chế và hướng phát triển

Hạn chế thiết kế cốt lõi trong Module 1 nằm ở **cơ chế gom snapshot cảm biến cứng nhắc**. Vì Backend Node.js chỉ thực hiện ghi nhận dữ liệu vào MongoDB và kiểm tra tự động hóa khi bộ đệm `sensorSnapshotBuffer` có đầy đủ cả 3 thông số cảm biến môi trường gửi lên, hệ thống sẽ gặp lỗi treo cứng (deadlock) nếu một trong các cảm biến vật lý trên phần cứng bị hỏng hoặc ngừng gửi tin nhắn. Trong trường hợp đó, toàn bộ dữ liệu môi trường sẽ không bao giờ được cập nhật tiếp vào cơ sở dữ liệu và các logic tự động hóa cảnh báo phía Backend cũng bị ngưng trệ hoàn toàn. Hướng khắc phục trong tương lai là áp dụng cơ chế timeout cho bộ đệm hoặc lưu trữ các thông số môi trường một cách độc lập kết hợp cơ chế nội suy dữ liệu cũ.

4.3 Module 2 — Tự động hóa và cảnh báo ngưỡng

4.3.1 Mục tiêu

Phát hiện các hành vi vi phạm ngưỡng an toàn môi trường dựa trên các tập cấu hình ngưỡng ngày và đêm; Tự động kích hoạt thiết bị tương ứng khi xảy ra vi phạm (như bật quạt khi trời nóng, bật đèn LED khi trời tối); Sinh các thông báo vi phạm hiển thị trên Dashboard và gửi cảnh báo khẩn cấp tới điện thoại của người dùng qua ứng dụng di động.

4.3.2 Trạng thái cài đặt

Hoàn thành một phần

4.3.3 Thành phần liên quan

Các thành phần mã nguồn tham gia trực tiếp bao gồm:

- **Gateway Python (Tự động hóa vật lý):** `controller.py` (dòng 90–113, 327) để phân luồng kiểm tra, `threshold_service.py` (dòng 142–192) để so sánh các quy tắc ngưỡng và phát lệnh Serial.

- **Backend Node.js (Cảnh báo an toàn):** `mqttService.js` (dòng 185–188) để kích hoạt kiểm tra khi đủ cảm biến, `alertService.js` (dòng 183–252) để xử lý logic sinh cảnh báo và ngăn cảnh báo trùng lặp.
- **Giao diện Dashboard:** `Dashboard.js` (dòng 13–18, 151–183) hiển thị khu vực cảnh báo nổi bật cùng nút bấm giải tỏa thủ công.
- **Tập cấu hình hệ thống:** `config.yml` (dòng 49–70) định nghĩa các quy tắc ngưỡng cứng ở Gateway và `alertService.js` (dòng 33–44) định nghĩa các ngưỡng tính Backend.

4.3.4 Luồng xử lý

Luồng tự động hóa vật lý và cảnh báo ngưỡng an toàn được thực hiện đồng thời qua hai tuyến xử lý như được biểu diễn trong Figure 3:

- **Tuyến tự động hóa cục bộ ở Gateway:**
 - Khi nhận được số đo từ cảm biến gửi lên qua Serial (ví dụ: nhiệt độ 32°C), `MainController` của Gateway lập tức chuyển dữ liệu tới `ThresholdService`.
 - `ThresholdService` duyệt qua tập cấu hình rules trong `config.yml`. Đối chiếu nhiệt độ $32 > 30$ (ngưỡng trên của quạt Fan), hệ thống xác định quy tắc bị vi phạm.
 - Để tránh gửi lệnh vật lý dồn dập, Gateway so sánh trạng thái trước đó. Nếu quạt đang tắt, Gateway sinh khung lệnh `!F:1#` gửi xuống Yolo:Bit để bật quạt vật lý và lưu vết vào Database.
- **Tuyến sinh cảnh báo an toàn trên Backend Express:**
 - Sau khi gom đủ snapshot cảm biến thành công, Backend gọi hàm kiểm tra `alertService.checkAndAlert`.
 - Hàm này so sánh giá trị cảm biến thực tế với cấu hình ngưỡng trong tệp tin. Nếu phát hiện vi phạm, hệ thống kiểm tra trong bộ sưu tập `Alert` xem có cảnh báo cùng loại nào đang mở (`isResolved: false`) hay không.
 - Nếu chưa tồn tại cảnh báo chủ động, Backend Express chèn bản ghi cảnh báo mới vào MongoDB, đồng thời lưu nhật ký dấu vết tự động `ThresholdTrace`.
 - Trình duyệt của người dùng thực hiện polling API `GET /api/alerts/active` mỗi 5 giây và hiển thị hộp thông tin màu đỏ cảnh báo trên màn hình.
 - Người dùng nhấn nút "Xác nhận" để gửi lệnh `PATCH /api/alerts/:id/resolve` lên Backend, cập nhật trạng thái `isResolved = true` trên MongoDB, làm biến mất hộp cảnh báo trên Dashboard.

4.3.5 Giao diện người dùng

Trình duyệt người dùng kết xuất thành phần "Cảnh báo ngưỡng" nổi bật có màu đỏ ở phía dưới khu vực hiển thị các thẻ cảm biến môi trường trên Dashboard. Mỗi thông báo vi phạm chứa tên cảm biến được dịch hóa hoàn toàn sang tiếng Việt (ví dụ: "Nhiệt độ", "Độ ẩm", "Ánh sáng") cùng mô tả chi tiết giá trị thực đo so

với ngưỡng an toàn quy định và có một nút bấm màu đỏ nhãn "Xác nhận" để giải tỏa cảnh báo (Tham chiếu: Figure 6).

4.3.6 Hạn chế và hướng phát triển

Module 2 hiện tại tồn tại hai khoảng trống thiết kế vô cùng nghiêm trọng cần ghi nhận một cách thực chứng:

- **Hoàn toàn thiếu tự động điều chỉnh ngưỡng theo ngày/đêm:** Hệ thống chỉ sử dụng một tập cấu hình ngưỡng tĩnh hoạt động cố định từ tệp tin cấu hình. Mã nguồn hoàn toàn không chứa bất kỳ logic kiểm tra múi giờ, lấy thời gian thực từ hệ thống hay lập lịch (cron job) để chuyển đổi qua lại giữa hai chế độ cấu hình ngày/đêm.
- **Hoàn toàn thiếu cảnh báo qua ứng dụng di động:** Không có bất kỳ dòng mã nào liên quan đến các nhà cung cấp dịch vụ gửi thông báo đẩy di động như Firebase Cloud Messaging (FCM), OneSignal hay tin nhắn SMS Twilio. Hệ thống hoàn toàn phụ thuộc vào việc polling giao diện web, điều này làm mất đi khả năng thông báo tức thời của một hệ thống an toàn chuyên nghiệp.

Hướng phát triển tiếp theo là xây dựng luồng thời gian thực cho phép Gateway tải lại cấu hình ngưỡng dựa trên bộ lập lịch giờ tự động và tích hợp gói SDK FCM của Firebase vào Backend Node.js để gửi thông báo khẩn tới điện thoại người dùng.

4.4 Module 3 — Giao diện điều khiển thiết bị

4.4.1 Mục tiêu

Thiết kế giao diện người dùng trên ứng dụng Web cho phép người dùng giám sát trạng thái và chủ động bật/tắt thủ công từ xa các thiết bị điện trong gia đình bao gồm đèn chiếu sáng (LED), quạt thông gió (Fan) và động cơ rèm cửa (Servo).

4.4.2 Trạng thái cài đặt

Đã hoàn thành

4.4.3 Thành phần liên quan

Các thành phần mã nguồn tham gia điều phối trực tiếp luồng bật/tắt thiết bị bao gồm:

- **Trình duyệt Frontend:** `DeviceManagement.js` (dòng 46–77, 143–170) và `api.js` (dòng 25–39) chịu trách nhiệm kết xuất giao diện thẻ bật/tắt và gửi yêu cầu REST.
- **Backend Express:** `device.js` (router POST `/api/devices/control`), `deviceController.js` (dòng 24–118) và `mqttService.js` (dòng 309–339) tiếp nhận yêu cầu, cập nhật MongoDB, lưu vết lịch sử điều khiển và đẩy lệnh lên MQTT.
- **Gateway Python:** `controller.py` (dòng 215–290) tiếp nhận thông tin điều khiển từ MQTT, giải mã và viết trực tiếp xuống cổng truyền Serial vật lý.

4.4.4 Luồng xử lý

Luồng điều khiển thiết bị thủ công được thực thi theo 7 bước tuần tự tương thích với lược đồ tuần tự của Figure 2:

- Người dùng nhấp chuột vào công tắc gạt trạng thái trên thẻ thiết bị (ví dụ: LED) trên trang Quản lý thiết bị của trình duyệt.
- Trình duyệt lập tức gửi yêu cầu HTTP POST đồng bộ tới Backend thông qua endpoint `/api/devices/control` chứa payload `{deviceName: "led", action: "on", deviceType: "led"}`.
- Backend Express nhận yêu cầu, tiến hành kiểm tra nghiệp vụ và ghi nhận bản ghi nhật ký điều khiển thủ công vào cơ sở dữ liệu MongoDB dưới dạng một tài liệu `ControlTrace`.
- Backend gọi hàm `mqttService.sendDeviceCommand` để xuất bản bản tin lệnh JSON `{"action": "on"}` tới topic MQTT `home/livingroom/device/led/set`. Đồng thời, Backend cập nhật tạm thời trạng thái thiết bị LED thành "on" trong snapshot gần nhất để giao diện đồng bộ tức thời.
- Gateway Python lắng nghe topic lệnh qua cầu nối `mqtt_client.py`, gọi hàm `_on_mqtt` trong `MainController` để dịch chuyển trạng thái điều khiển thành giá trị nhị phân kỹ thuật (`on` → 1).
- Gateway truyền khung ký tự Serial `!L:1#` xuống Kit Yolo:Bit thông qua cổng UART. Kit Yolo:Bit nhận lệnh, tác động Relay cấp nguồn cho đèn sáng vật lý và phản hồi ngược lại chuỗi `!L:1#` qua UART để xác nhận.
- Gateway Python tiếp nhận phản hồi, đẩy trạng thái chính thức lên topic MQTT `home/livingroom/device/led/state` để Backend Express thu thập và đồng bộ snapshot thiết bị lâu dài vào cơ sở dữ liệu MongoDB.

4.4.5 Giao diện người dùng

Trang Quản lý thiết bị (`/devices`) hiển thị nhãn tiêu đề "Các thiết bị được kết nối" và kết xuất lưới các thẻ điều khiển thiết bị bao gồm Đèn LED, Quạt và Rèm cửa động cơ Servo. Mỗi thẻ thiết bị hiển thị biểu tượng trạng thái cùng một công tắc gạt ngang để người dùng thao tác. Khi thiết bị được bật thành công, thẻ chuyển màu nền sáng nổi bật kèm nhãn hiển thị trạng thái "Active" (Tham chiếu: Figure 7).

4.4.6 Hạn chế và hướng phát triển

Module 3 hoạt động ổn định và có độ trễ phản hồi phần cứng cực thấp nhờ sự kết hợp giữa MQTT và truyền tải Serial UART. Không phát hiện khoảng trống thiết kế nghiêm trọng nào ảnh hưởng đến trải nghiệm điều khiển thủ công cơ bản. Hướng phát triển tiếp theo là bổ sung cơ chế xác thực mã hóa hai đầu cho khung truyền Serial để ngăn chặn các cuộc tấn công tiêm lệnh trái phép từ các thiết bị Serial giả mạo.

4.5 Module 4 — Lưu trữ lịch sử và ghi sự kiện

4.5.1 Mục tiêu

Lưu trữ bền vững lịch sử hoạt động đo đạc của cảm biến môi trường và trạng thái thiết bị theo thời gian; Ghi nhật ký chi tiết các sự kiện vận hành hệ thống bao gồm: tương tác thủ công của người dùng trên Web, quá trình xử lý giọng nói và các tác động tự động hóa do vi phạm ngưỡng kích hoạt.

4.5.2 Trạng thái cài đặt

Hoàn thành một phần

4.5.3 Thành phần liên quan

Các cấu trúc lưu trữ và mã nguồn ghi sự kiện bao gồm:

- **Cơ sở dữ liệu lưu trữ snapshot:** `mqttService.js` (dòng 138, 178) thực hiện gọi dịch vụ lưu trữ dữ liệu môi trường và thiết bị định kỳ vào MongoDB.
- **Nhật ký tương tác người dùng (Manual & Voice):** `deviceController.js` (dòng 93–100) và `voiceService.js` (dòng 210, 217) gọi cơ chế chèn vết `ControlTrace`.
- **Nhật ký tự động hóa (Threshold logic):** `alertService.js` (dòng 239–250) thực hiện gọi chèn vết tự động `ThresholdTrace`.
- **Cấu trúc Schema MongoDB:** `Sensor.js`, `Device.js`, `ControlTrace.js` và `ThresholdTrace.js` định nghĩa cấu trúc bảng dữ liệu.

4.5.4 Luồng xử lý

Hệ thống lưu giữ lịch sử hoạt động bền vững thông qua 4 quy trình chạy nền liên tục:

- **Ghi lịch sử cảm biến định kỳ:** Khi Gateway Python đăng tải các số đo thô lên MQTT và Backend Express gom đủ snapshot môi trường thành công, hệ thống tự động ghi bản ghi mới vào bộ sưu tập `sensors` kèm nhãn thời gian thực tế.
- **Lưu vết tương tác thủ công:** Mọi yêu cầu HTTP POST gửi tới `deviceController` đều được xử lý và chèn một tài liệu mới vào bộ sưu tập `control_traces` chứa các trường thông tin: `source`: "frontend", tên action lệnh, `payload` truyền lên, và cờ `status` báo cáo thành công hay thất bại.
- **Lưu vết điều khiển giọng nói:** Khi người dùng ra lệnh bằng tiếng nói, dịch vụ `voiceService.js` ghi nhận chuỗi văn bản dịch nghĩa và intent ML tương ứng chèn vào bộ sưu tập `control_traces` với `source`: "api-client".
- **Ghi vết tự động hóa do vượt ngưỡng:** Khi Gateway hoặc Backend thực thi lệnh khẩn cấp kích hoạt thiết bị để bảo vệ hệ thống do vi phạm ngưỡng an toàn (ví dụ: bật quạt khi nóng quá mức),

`alertService.js` chèn nhật ký vào bộ sưu tập `threshold_traces` lưu thông tin về loại cảm biến vi phạm, giá trị đo đạc, ngưỡng an toàn và trạng thái thiết bị bị thay đổi.

4.5.5 Giao diện người dùng

Hệ thống hoàn toàn không có giao diện người dùng phục vụ chức năng xem lịch sử hay sự kiện. Ứng dụng Web chỉ cung cấp trang Dashboard hiện hành và trang Thiết bị, thanh điều hướng Sidebar hoàn toàn thiếu các thẻ liên kết để mở trang Xem đồ thị lịch sử cảm biến, Lịch sử cảnh báo hay Nhật ký hệ thống (log viewer). Người dùng không thể xem lại các dấu vết lịch sử nếu không truy cập trực tiếp vào MongoDB thông qua các công cụ kỹ thuật như MongoDB Compass.

4.5.6 Hạn chế và hướng phát triển

Hạn chế thiết kế vô cùng lớn của Module 4 là **sự thiếu vắng hoàn toàn của giao diện tương tác lịch sử và API truy vấn lịch sử**. Mặc dù mã nguồn Backend chèn các dấu vết `ControlTrace` và `ThresholdTrace` cực kỳ đầy đủ vào cơ sở dữ liệu MongoDB, hệ thống lại hoàn toàn thiếu các API phục vụ tìm kiếm phân trang dữ liệu lịch sử và thiếu giao diện hiển thị (như trang báo cáo, biểu đồ đường lịch sử biến thiên nhiệt độ/độ ẩm Recharts trên Web). Người dùng thông thường không có cách nào tiếp cận các thông tin lịch sử hoạt động hữu ích này. Hướng phát triển là bổ sung các API phân trang tại Backend Express và thiết kế thêm màn hình Lịch sử tích hợp biểu đồ trực quan giúp người dùng theo dõi và xuất báo cáo.

4.6 Module 5 — Điều khiển bằng giọng nói và gợi ý thông minh

4.6.1 Mục tiêu

Tích hợp khả năng ghi âm giọng nói tiếng Việt từ giao diện người dùng, sử dụng các dịch vụ trí tuệ nhân tạo để dịch âm thanh thành văn bản và phân loại ý định để điều khiển thiết bị; Đồng thời, tự động phân tích dữ liệu môi trường để đưa ra các gợi ý đề xuất đóng/mở rèm cửa và cửa sổ thông qua mô hình cây quyết định Decision Tree.

4.6.2 Thành phần liên quan

Các thành phần và mã nguồn xử lý luồng AI bao gồm:

- **Giao diện Web (PTT):** `VoiceControl.js` (dòng 14–25, 72–79, 81–97) xử lý thu âm bằng `RecordRTC` và hiển thị Toast phản hồi kết quả giọng nói.
- **Backend Express:** `voiceRoutes.js` (POST `/api/voice/command`), `voiceService.js` (dòng 162–220) điều phối luồng gọi AI và xuất bản lệnh MQTT.
- **Dịch vụ phụ trợ AI:** `vosk_server.py` (dịch vụ STT cổng 8500) và `ml_server.py` (dịch vụ phân loại intent cổng 8000).
- **Mô hình đề xuất tự động (Gateway):** `ai_service.py` (dòng 16–174) xử lý chạy mô hình và `curtain_control_system.py` huấn luyện mô hình cây quyết định Decision Tree (`curtain_model.pkl`).

4.6.3 Luồng xử lý

Luồng xử lý điều khiển bằng giọng nói tiếng Việt được thực hiện theo 7 bước nghiêm ngặt được mô tả tại lược đồ tuần tự của Figure 4:

- Người dùng nhấn và giữ phím biểu tượng Mic ở chân màn hình Web, nói rõ câu lệnh tiếng Việt (ví dụ: "bật đèn") và nhả chuột.
- Trình duyệt sử dụng công cụ RecordRTC lưu luồng ghi âm định dạng mono WAV tần số lấy mẫu chuẩn 16kHz gửi POST lên Backend Express qua API `/api/voice/command` dưới dạng multipart.
- Backend tiếp nhận dữ liệu nhị phân của file âm thanh, chuyển luồng binary này gọi POST tới dịch vụ STT Vosk hoạt động tại cổng `8500/transcribe` để nhận diện chuỗi văn bản tiếng Việt.
- Nhận được chuỗi văn bản tiếng Việt từ Vosk (ví dụ: "bật đèn"), Backend Express tiếp tục gửi yêu cầu POST chứa chuỗi văn bản này tới dịch vụ ML Intent hoạt động tại cổng `8000/predict`.
- Mô hình học máy phân loại văn bản sang định dạng intent kỹ thuật (ví dụ: `led:on`), Backend chuẩn hóa và ánh xạ thành công lệnh điều khiển tương ứng (`device: "led", action: "on"`).
- Backend thực hiện lưu trữ thông tin sự kiện vào cơ sở dữ liệu MongoDB và xuất bản lệnh điều khiển lên Broker MQTT thông qua chủ đề `home/default/device/led/set`.
- Trình duyệt nhận phản hồi JSON từ Backend và hiển thị một hộp Toast đen mờ trong 5 giây ghi nhận văn bản nhận diện được và ý định hệ thống đang thực thi.

4.6.4 Giao diện người dùng

Giao diện điều khiển bằng giọng nói được triển khai dưới dạng một nút Microphone trôi nổi (Float component) luôn xuất hiện ở góc dưới bên phải màn hình Web Layout. Khi người dùng nhấp giữ chuột, biểu tượng Micro màu xanh chuyển sang hình ô vuông màu đỏ nhấp nháy phát xung lan tỏa thể hiện trạng thái đang ghi âm. Khi nhả chuột và Backend đang xử lý, nút chuyển sang biểu tượng xoay tải. Sau đó, một hộp Toast kết quả xuất hiện trên góc màn hình ghi nhận chi tiết câu lệnh người dùng nói và ý định hệ thống hiểu được (Tham chiếu: Figure 8).

4.6.5 Hạn chế và hướng phát triển

Module 5 hiện tại bộc lộ hai khoảng trống lớn và một lỗi thiết kế nghiêm trọng cản trở quá trình vận hành thực tế:

- **Lỗi không khớp chủ đề điều khiển vị trí (Location Mismatch):** Đây là một lỗi thiết kế nghiêm trọng trong mã nguồn Backend. Khi dịch nghĩa giọng nói thành công và xuất bản lệnh điều khiển lên MQTT, Backend sử dụng tiền tố phân vùng vị trí mặc định là `default` (tạo ra chủ đề điều khiển là `home/default/device/led/set`). Tuy nhiên, Gateway Python lại được cấu hình cứng lắng nghe các tin

nhấn điều khiển thuộc phòng khách (`home/livingroom/device/led/set` cấu hình trong `config.yml`). Sự không đồng nhất này khiến cho lệnh điều khiển từ giọng nói dù nhận diện thành công vẫn bị bỏ qua trên Gateway và không bao giờ bật được thiết bị phần cứng vật lý.

- **Chưa kích hoạt AI điều khiển rèm tự động:** Dù Gateway đã phát triển thuật toán chạy mô hình cây quyết định Decision Tree (`ai_service.py`) sử dụng tệp mô hình `curtain_model.pkl` để tự động ra quyết định điều khiển rèm dựa trên nhiệt độ, độ ẩm và ánh sáng, tính năng này hoàn toàn bị vô hiệu hóa trong luồng chạy thực tế do cấu hình `automation.ai.enabled` bị gán cứng là `false` trong `config.yml`.
- **Thiếu giao diện hiển thị gợi ý thông minh:** Hệ thống hoàn toàn không phát triển giao diện Web hỗ trợ hiển thị các hộp thoại gợi ý đề xuất điều khiển đóng/mở rèm cửa và cửa sổ dựa trên môi trường sống để người dùng phê duyệt hay từ chối, vi phạm trực tiếp yêu cầu thiết kế ban đầu.

Hướng phát triển tiếp theo là đồng bộ hóa vị trí mặc định giữa Backend Express và Gateway Python (sửa đổi về chung phòng `livingroom`), kích hoạt cờ tự động hóa AI và thiết kế giao diện hiển thị thể thông báo gợi ý thông minh đề xuất cho người dùng trên Web Dashboard.

4.7 Thiết kế AI: Điều khiển bằng giọng nói

4.8 Thiết kế AI: Gợi ý thông minh dựa trên dữ liệu môi trường

4.8.1 Xây dựng tập dữ liệu (Dataset)

Phương pháp tạo dữ liệu: Khác với các bài toán có sẵn tập dữ liệu được gán nhãn thực tế, hệ thống đề xuất điều khiển rèm thông minh sử dụng phương pháp sinh dữ liệu tổng hợp (synthetic data generation) dựa trên một bộ tập luật (heuristics/rules). Bộ quy tắc này mô phỏng hành vi điều chỉnh rèm của người dùng thực tế nhằm tối ưu hóa ánh sáng tự nhiên và ổn định nhiệt độ phòng. Các trường dữ liệu môi trường sẽ được sinh ngẫu nhiên tuân theo các phân bố thống kê cụ thể để mô phỏng sự đa dạng của môi trường thực:

- Ánh sáng (`sunlight_percent`): Dùng phân bố chuẩn $N(\mu = 50, \sigma = 25)$ (giới hạn trong khoảng 0–100%).
- Nhiệt độ (`temperature`): Dùng phân bố đều $U(-10, 50)$ (°C).
- Độ ẩm (`humidity_percent`): Dùng phân bố chuẩn $N(\mu = 60, \sigma = 20)$ (giới hạn trong khoảng 0–100%).

Mỗi tổ hợp dữ liệu (X) được sinh ra sẽ tự động được gán nhãn hành động y (0: Đóng rèm, 1: Mở rèm) dựa vào sự kết hợp của các điều kiện. Ví dụ: Nếu ánh sáng $> 90\%$ hoặc (nhiệt độ $\geq 33^\circ\text{C}$ và ánh sáng $> 70\%$) thì đóng rèm (tránh chói và giảm hấp thụ nhiệt). Các trường hợp còn lại như nhiệt độ thấp, độ ẩm cao, hoặc ánh sáng không quá gắt đều hướng đến việc mở rèm để tận dụng dưỡng khí và nguồn sáng tự nhiên.

Thống kê tập dữ liệu:

- Lượng mẫu sinh ra: Hệ thống tự động sinh hai tập dữ liệu độc lập lưu dưới định dạng CSV, bao gồm Tập huấn luyện (`curtain_train.csv`) với **10.000 mẫu** và tập dữ liệu kiểm thử (`curtain_test.csv`) gồm **2.000 mẫu**.

- **Tỉ lệ Train/Validation/Test:** Để huấn luyện, tập dữ liệu 10.000 mẫu phân tách theo tỷ lệ 80/20, với 8.000 mẫu dành cho Huấn luyện (Train set) và 2.000 mẫu dành cho quá trình Thử nghiệm (Validation set). Tập Test 2.000 mẫu đứng độc lập so với tập 10.000 ban đầu để đánh giá độc lập nhất.
- **Tỉ lệ phân phối nhãn:** Sự phân biệt giữa các quy tắc an toàn mang tính phòng vệ (đóng rèm khi điều kiện khắc nghiệt) gây ra sự thiếu cân bằng nhãn dạng nhẹ. Tại tập huấn luyện, có 9.008 mẫu mang nhãn Mở rèm (1) và 992 mẫu mang nhãn Đóng rèm (0) (Tỉ lệ xấp xỉ 9:1). Ở tập kiểm thử Test 2.000 mẫu, phân phối tương tự với 1.794 mẫu mở và 206 mẫu đóng.

4.8.2 Cấu trúc mô hình

Mô hình thông minh đóng nhãn tự động này sử dụng thuật toán phân loại **Decision Tree (Cây quyết định)**, thực thi trên nền tảng thư viện máy học tiêu chuẩn **scikit-learn** của Python. Lý do cho việc ứng dụng thuật toán có trọng số nhẹ như Cây phân lớp thay vì những mạng Deep Learning phức tạp là vì tính minh bạch (khả năng giải thích tại sao mô hình ra hành động), vận hành siêu nhanh với những tài nguyên chặt hẹp, và bản chất bài toán được định nghĩa qua một loạt nguyên tắc luật cứng - nơi cây quyết định có thể phản ánh hoàn toàn chính xác luồng quy tắc tư duy đó nhờ quá trình phân nhánh đồ thị logic.

Mô hình được định nghĩa bằng lớp **DecisionTreeClassifier** với các tham số được cấu hình tỉ mỉ giúp giảm thiểu hiện tượng Overfitting trên các nhiễu thống kê:

- **max_depth = 10:** Không chế độ sâu tối đa của cây ở mức 10. Nếu cây phát triển vô hạn, nó có thể ghi nhớ máy móc từng con số nhỏ ở luật do việc rẽ nhánh hỗn loạn. Độ sâu này thiết lập tính tổng quát hóa cao cho các mẫu cảm biến trong tương lai.
- **min_samples_split = 20:** Một nút trung gian cần ít nhất 20 mẫu dữ liệu tập trung mới được thuật toán quyền rẽ đôi nhánh. Tránh chẻ nhánh tại các trường hợp rác hoặc quá nhỏ lẻ.
- **min_samples_leaf = 10:** Chỉ cho phép một lá kết luận tồn tại khi ôm trọn tối thiểu 10 quan sát. Ngăn hình thành những lá lẻ loi để bao lấy một biến thiên duy nhất.
- **random_state = 42:** Tham số cung cấp tính ổn định trong việc cố định mầm ngẫu nhiên, giúp bất kể khi nào cây quyết định được tạo lại thì cách rẽ nhánh của cây đều hội tụ nhất quán về một đồ thị và không thay đổi theo từng lần chạy.

4.8.3 Đánh giá kết quả (Evaluation)

Chất lượng học luật của mô hình Decision Tree được kiểm thử và đánh giá tổng thể thông qua *Confusion Matrix* và *Classification Report* (độ chính xác Accuracy) trên cả tập Validation (2.000 dòng) và tập Kiểm thử độc lập (2.000 dòng). Vì dữ liệu đầu vào vốn dĩ được sinh cấu trúc hóa từ một bản nội quy If-Else chặt chẽ, thuật toán Cây Quyết định nhanh chóng biểu diễn lại được cấu trúc tư duy này, dẫn tới độ chính xác (Accuracy Score) hội tụ ở mức cực kỳ cao ($> 99.00\%$) đi kèm tỷ lệ f1-score và recall gần như tuyệt đối giữa hai trạng thái Nhãn 0 và Nhãn 1.

4.8.4 Hướng phát triển

Mặc dù đạt được độ chính xác khởi điểm xuất sắc, cần nhấn mạnh rằng bộ dữ liệu tổng hợp 10.000 mẫu ban đầu chỉ đóng vai trò "**khởi động lạnh**" (**cold-start**). Mục tiêu của tập sinh ngẫu nhiên định sẵn là giúp cấu trúc cây rẽ nhánh có đủ nền tảng kiến thức thô để hoạt động hợp lý ngay khi vừa triển khai thiết bị, cung cấp các gợi ý mặc định an toàn cho môi trường. Tuy nhiên, bản chất cốt lõi của Machine Learning trong YOLOHome là hướng tới khả năng tự tinh chỉnh dựa theo lối sống sinh hoạt đặc thù của con người.

Để hiện thực hóa hoàn toàn ý tưởng đó, hướng phát triển kế tiếp là tích hợp bản thân mô hình hiện tại vào một pipeline **Học và Huấn luyện liên tục (Continuous Learning)** với các luồng dữ liệu thực tế:

- **Thu thập mẫu theo thời gian thực:** Thay vì sinh dữ liệu ngẫu nhiên, hệ thống sẽ lắng nghe lịch sử "Bật/Tắt rèm thủ công" trực tiếp từ người dùng thông qua Dashboard. Nếu người dùng bác bỏ một gợi ý tự động (ví dụ: tự động đóng rèm nhưng người dùng lại ghi đè công tắc mở lên), hệ thống chèn sự kiện này kèm theo snapshot môi trường tương ứng (Độ ẩm, nhiệt độ, ánh sáng) vào cơ sở dữ liệu MongoDB dưới tư cách là cụm nhãn vàng (ground truth label).
- **Huấn luyện định kỳ:** Tận dụng các cron job lập lịch chạy ngầm ở Gateway, cứ cuối mỗi tuần hệ thống sẽ kéo toàn bộ cụm nhãn vàng nói trên từ MongoDB xuống, nạp nối tiếp vào dữ liệu cũ. Cây quyết định được tái huấn luyện (retrain) hoặc cập nhật trọng số để dần uốn nắn cấu trúc rẽ nhánh khớp với thói quen sinh hoạt (VD: chủ nhà thích phòng lạnh hơn quy định chuẩn).

4.9 Yolo:Bit

4.9.1 1. Giai đoạn khởi tạo (Initialization)

- **Khởi tạo phần cứng:** Kích hoạt và cấu hình các module ngoại vi bao gồm cảm biến nhiệt độ - độ ẩm (DHT20), màn hình LCD1602 (giao tiếp I2C), LED RGB, Quạt (tín hiệu Analog) và động cơ Servo.
- **Khởi tạo trạng thái:** Thiết lập toàn bộ thiết bị chấp hành (Actuators) về trạng thái tắt (0) hoặc góc quay mặc định để đảm bảo an toàn khi hệ thống vừa cấp nguồn. Màn hình LED ma trận hiển thị thông báo "DADN" và "OK" để xác nhận khởi tạo thành công.

4.9.2 2. Luồng thu thập dữ liệu và Giao tiếp Serial (Main Loop)

Đây là vòng lặp vô tận (`while True`) đóng vai trò duy trì hoạt động của hệ thống, thực hiện các tác vụ với tần suất cao:

- **Đọc cảm biến liên tục:** Cập nhật giá trị nhiệt độ (RT), độ ẩm (RH) từ DHT20 và đo cường độ ánh sáng (LUX) qua chân Analog, sau đó map giá trị ánh sáng về dải phần trăm (0–100%).
- **Quét tín hiệu đầu vào (Polling Input):** Sử dụng `uselect.poll()` để kiểm tra non-blocking bộ đệm Serial. Nếu có chuỗi lệnh gửi xuống từ Gateway, hệ thống sẽ đọc và tiến hành bóc tách (parse) chuỗi lệnh.

- **Phân rã cú pháp (Parsing):** Lệnh được quy ước theo cú pháp `!<Device>:<Status>#`. Hệ thống sẽ tách chuỗi để xác định thiết bị mục tiêu (`device`) và lệnh điều khiển (`status`).

4.9.3 3. Luồng xử lý sự kiện (Event Manager)

Thay vì dùng các lệnh `if-else` lồng nhau phức tạp để kiểm tra chuỗi lệnh, hệ thống sử dụng `event_manager` để đăng ký các bộ lắng nghe (`listeners`), giúp code module hóa và dễ mở rộng:

- **Sự kiện điều kiện (Condition Events):** Khi có lệnh mới được parse thành công từ Serial, hệ thống đối chiếu prefix của thiết bị:
 - `!L::` Kích hoạt thay đổi biến trạng thái của đèn LED.
 - `!F::` Kích hoạt thay đổi biến trạng thái của quạt (băm xung PWM thay đổi tốc độ).
 - `!S::` Kích hoạt thay đổi biến trạng thái của Servo (điều khiển góc quay 0 hoặc 180 độ).
- **Thực thi phần cứng:** Các hàm `control_led()`, `control_fan()`, `control_servo()` được gọi ở cuối vòng lặp để áp đặt mức tín hiệu điện áp thực tế ra các chân GPIO dựa trên các biến trạng thái vừa được cập nhật.

4.9.4 4. Luồng đồng bộ trạng thái định kỳ (Timer Event)

- Hệ thống đăng ký một `timer_event` chu kỳ 1000 ms (1 giây).
- **Chức năng:** Đóng gói toàn bộ giá trị cảm biến hiện tại (Nhiệt độ, Độ ẩm, Ánh sáng) và trạng thái thực tế của các thiết bị (Đèn, Quạt, Servo) thành các Data Frames chuẩn (VD: `!T:25.5# !H:60# !L:1#`).
- **Gửi lên Gateway:** Đẩy các chuỗi này qua cổng Serial để Gateway nắm bắt trạng thái (Telemetry) và hiển thị trực tiếp các thông số lên màn hình LCD1602 cục bộ để thuận tiện cho việc giám sát trực tiếp.

4.10 Các kênh giao tiếp

Để đảm bảo các dịch vụ hoạt động thống nhất trong hệ thống, thông tin chi tiết về các kênh truyền thông bao gồm REST API endpoints, các chủ đề (topics) MQTT và giao thức truyền Serial UART được định nghĩa rõ ràng.

Dưới đây là danh mục chi tiết các kênh truyền thông được trích xuất trực tiếp từ mã nguồn thực tế của dự án.

Bảng 2: Danh mục chi tiết các kênh truyền thông trong hệ thống YOLOHome

Giao thức	Endpoint / MQTT Topic	Cấu trúc dữ liệu (Payload)	Tệp nguồn liên quan
HTTP REST	POST /api/users/signup	{ username, password, fullName }	userController.js:5

Tiếp tục trang sau

Giao thức	Endpoint / MQTT Topic	Cấu trúc dữ liệu (Payload)	Tệp nguồn liên quan
HTTP REST	POST /api/users/login	{ username, password }	UserController.js:43
HTTP REST	GET /api/sensors/latest	Trả về chuỗi JSON thông số cảm biến gần nhất	sensorController.js:5
HTTP REST	GET /api/devices/latest	Trả về chuỗi JSON trạng thái thiết bị gần nhất	deviceController.js:11
HTTP REST	POST /api/devices/control	{ deviceName, action, deviceType }	deviceController.js:24
HTTP REST	GET /api/alerts/active	Trả về danh sách cảnh báo chưa xử lý	alertController.js:23
HTTP REST	PATCH /api/alerts/:id/resolve	Đánh dấu cảnh báo đã được xác nhận xử lý	alertController.js:36
HTTP REST	POST /api/voice/command	File ghi âm dạng Multipart với trường audio	voiceRoutes.js:16
HTTP Client	POST /transcribe	Gửi file WAV (multipart) cổng 8500 sang Vosk	voskProvider.js:10
HTTP Client	POST /predict	{ text: transcript } cổng 8000 sang ML	voiceService.js:179
MQTT	home/{room}/device/{type}/set	{"action": "on" "off"}	mqttTopics.js:20
MQTT	home/{room}/device/{type}/state	{"action": "on" "off"}	mqttTopics.js:21
MQTT	home/{room}/sensor/{type}	{"value": "<numeric_string>"}	mqttTopics.js:23
MQTT	home/system/getall	{}	mqttTopics.js:10
MQTT	home/system/stateall	{ "temp": X, "humi": Y, "light": Z, ... }	mqttTopics.js:11
Serial UART	Cổng kết nối vật lý RX/TX	Khung truyền dạng !<ABBR>:<VALUE>#	default_adapter.py:10

Ngoài ra, nhằm quản lý luồng thông điệp bất đồng bộ của mạng MQTT một cách hiệu quả, hệ thống định nghĩa các chủ đề MQTT theo cấu trúc phân cấp rõ ràng được trình bày trong Table 3.

Bảng 3: Định nghĩa cấu trúc các chủ đề MQTT trong hệ thống

Chủ đề (Topic) MQTT	Chiều truyền tin	Ý nghĩa hoạt động trong hệ thống
home/+ /device/+ /set	Backend → Gateway	Gửi yêu cầu điều khiển thiết bị (Bật/Tắt quạt, đèn, servo)
home/+ /device/+ /state	Gateway → Backend	Cập nhật phản hồi trạng thái thực tế sau khi thiết bị đổi trạng thái
home/+ /sensor/+	Gateway → Backend	Đẩy các giá trị số đo nhiệt độ, độ ẩm, ánh sáng thô
home/system/getall	Backend → Gateway	Yêu cầu Gateway truy vấn toàn bộ trạng thái hệ thống phần cứng
home/system/stateall	Gateway → Backend	Phản hồi snapshot toàn bộ thiết bị vật lý và cảm biến

Để chuyển đổi qua lại giữa mạng tin nhắn MQTT và cổng truyền nhận Serial kết nối trực tiếp Yolo:Bit, Gateway áp dụng quy ước bảng mã hóa viết tắt các thiết bị và cảm biến vật lý được liệt kê tại Table 4.

Bảng 4: Bảng ánh xạ ký tự viết tắt giao thức truyền nhận Serial

Tên vật lý	Mã kí tự	Loại thiết bị	Chiều truyền tin	Ví dụ khung truyền Serial
Nhiệt độ	T	Cảm biến	Kit → Gateway	!T:28.5# (Nhiệt độ đo được 28.5°C)
Độ ẩm	H	Cảm biến	Kit → Gateway	!H:65# (Độ ẩm đo được 65%)
Ánh sáng	Lu	Cảm biến	Kit → Gateway	!Lu:100# (Cường độ ánh sáng 100 lx)
Đèn LED	L	Chấp hành	Hai chiều	!L:1# (Bật LED) hoặc !L:0# (Tắt LED)
Quạt DC (Fan)	F	Chấp hành	Hai chiều	!F:1# (Bật quạt) hoặc !F:0# (Tắt quạt)
Động cơ Servo	S	Chấp hành	Hai chiều	!S:1# (Mở rèm) hoặc !S:0# (Đóng rèm)

Hệ thống cơ sở dữ liệu MongoDB lưu trữ dữ liệu bền vững của hệ thống thông qua các bộ sưu tập (collections) được tổ chức theo cấu trúc phi quan hệ, giúp quản lý lịch sử vận hành và hoạt động an toàn được mô tả chi tiết tại Table 5.

Bảng 5: Cấu trúc các bộ sưu tập MongoDB trong hệ thống YOLOHome

Tên bộ sưu tập	Model Schema	Mô tả chức năng lưu trữ dữ liệu
sensors	Sensor	Lưu trữ lịch sử snapshot dữ liệu môi trường (nhiệt độ, độ ẩm, ánh sáng)
devices	Device	Lưu trữ lịch sử snapshot trạng thái các thiết bị chấp hành (đèn, quạt, servo)
alerts	Alert	Lưu trữ các cảnh báo vượt ngưỡng an toàn môi trường chưa hoặc đã được xử lý
control_traces	ControlTrace	Ghi nhận vết các hoạt động điều khiển thiết bị thủ công và bằng giọng nói tiếng Việt
threshold_traces	ThresholdTrace	Ghi nhật ký lịch sử các hành động tự động hóa kích hoạt thiết bị do vượt ngưỡng
users	User	Lưu trữ thông tin tài khoản người dùng đăng nhập hệ thống

Quá trình cấu hình toàn bộ hệ thống khi triển khai thực tế trên môi trường container hóa (Docker) được quản lý thông qua tập hợp các biến môi trường được liệt kê đầy đủ tại Table 6.

Bảng 6: Danh mục các biến môi trường cấu hình hệ thống

Tên biến môi trường	Dịch vụ	Mô tả vai trò cấu hình trong hệ thống
PORT	backend	Cổng chạy dịch vụ Backend REST API Express (mặc định: 5000)
NODE_ENV	backend	Chế độ môi trường vận hành ứng dụng Node.js (development/production)
MONGO_URI	backend	Địa chỉ kết nối trực tiếp đến cơ sở dữ liệu MongoDB
CORS_ORIGIN	backend	Chính sách chia sẻ tài nguyên nguồn gốc chéo CORS (mặc định: *)
MQTT_ENABLED	backend	Bật/tắt việc kết nối mạng truyền điệp MQTT Broker (true/false)
MQTT_BROKER_URL	backend	Địa chỉ kết nối đến Broker Mosquitto (mặc định: mqtt://mqtt:1883)
VOSK_SERVER_URL	backend	URL kết nối đến dịch vụ STT nhận dạng giọng nói tiếng Việt Vosk
ML_SERVICE_URL	backend	URL kết nối đến dịch vụ ML phân loại ý định câu lệnh giọng nói
MONGO_INITDB_DATABASE	mongo	Tên cơ sở dữ liệu mặc định được khởi tạo tự động trong MongoDB
PYTHONUNBUFFERED	gateway	Bắt buộc tiến trình Python xuất trực tiếp log ra terminal, tránh buffer
PYTHONDONTWRITEBYTECODE	gateway	Ngăn chặn Python sinh ra các tệp mã máy trung gian .pyc trong container

Cuối cùng, ma trận trạng thái tích hợp của các module tính năng trong dự án giúp tác giả báo cáo tự đánh giá trực quan mức độ triển khai được tổng kết tại Table 7.

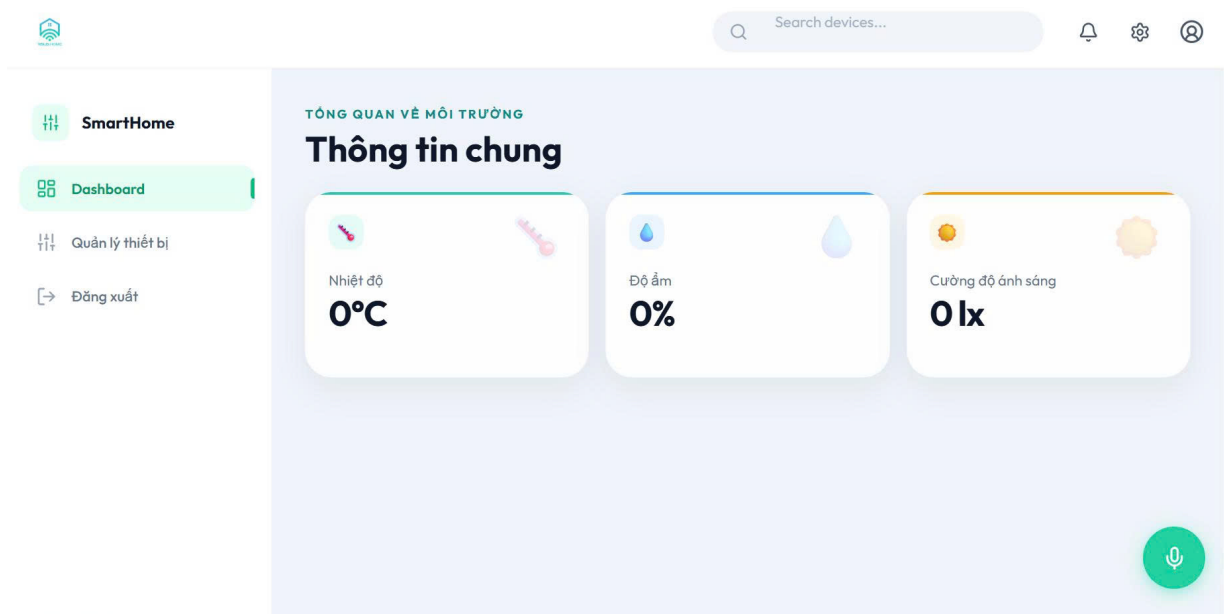
Bảng 7: Ma trận trạng thái phát triển các module hệ thống YOLOHome

Phân loại các module chức năng nghiệp vụ	Trạng thái cài đặt thực tế
Module 1 — Đo đạc và hiển thị môi trường	Đã hoàn thành
Module 2 — Tự động hóa và cảnh báo ngưỡng	Hoàn thành một phần
Module 3 — Giao diện điều khiển thiết bị	Đã hoàn thành
Module 4 — Lưu trữ lịch sử và ghi sự kiện	Hoàn thành một phần
Module 5 — Điều khiển bằng giọng nói và gợi ý thông minh	Hoàn thành một phần

5 Giao diện Website dự án

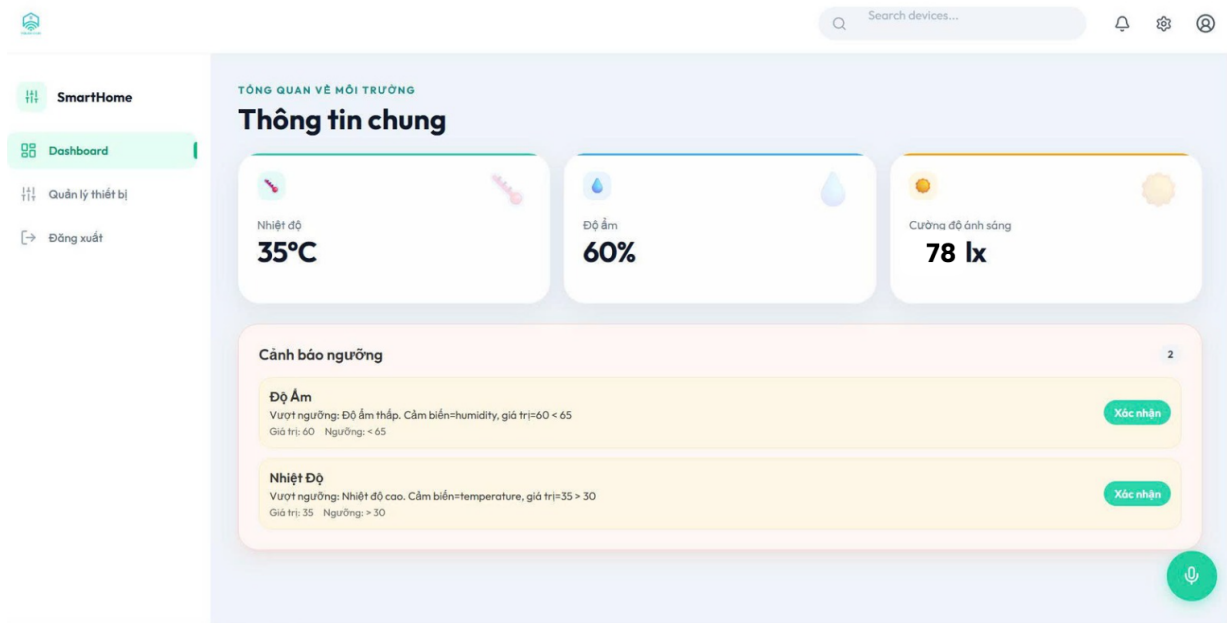
Để chứng minh kết quả phát triển giao diện hệ thống giám sát và điều khiển nhà thông minh YOLOHome, dưới đây là danh sách các ảnh chụp màn hình ghi nhận giao diện vận hành thực tế của hệ thống. Các tệp ảnh chụp giao diện được lưu trữ tại thư mục **figures/** dưới các tên tương ứng và được liên kết bằng các mã hình vẽ để dễ dàng kiểm tra.

Hệ thống cung cấp màn hình Đăng nhập (/) và màn hình Đăng ký (/signup) có thiết kế giao diện theo phong cách Sleek Dark-mode tối giản và trực quan, hỗ trợ người dùng bảo mật tài khoản cá nhân. Sau khi đăng nhập thành công, người dùng được điều hướng đến trang Dashboard hiển thị thông số cảm biến môi trường thời gian thực.



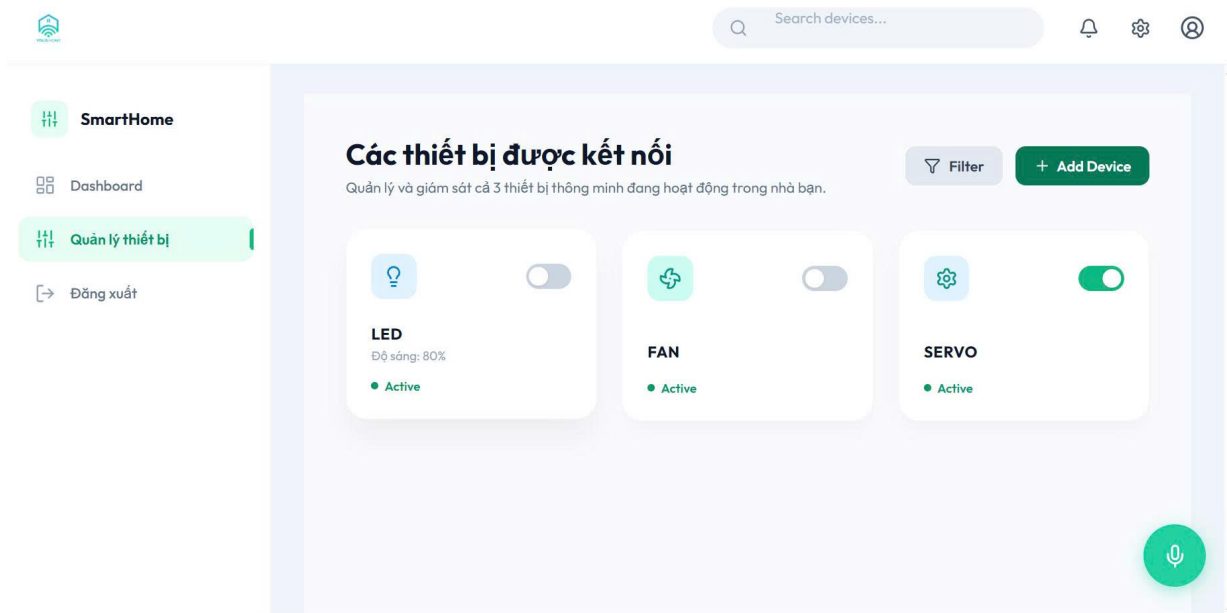
Hình 5: Ảnh chụp giao diện Dashboard hiển thị thông số cảm biến môi trường

Giao diện Dashboard tại Figure 5 cập nhật dữ liệu liên tục sau mỗi 5 giây. Khi phát hiện các chỉ số cảm biến vượt qua ngưỡng cấu hình an toàn, hệ thống tự động sinh cảnh báo và hiển thị hộp thông tin màu đỏ nổi bật trực quan như trình bày tại Figure 6.



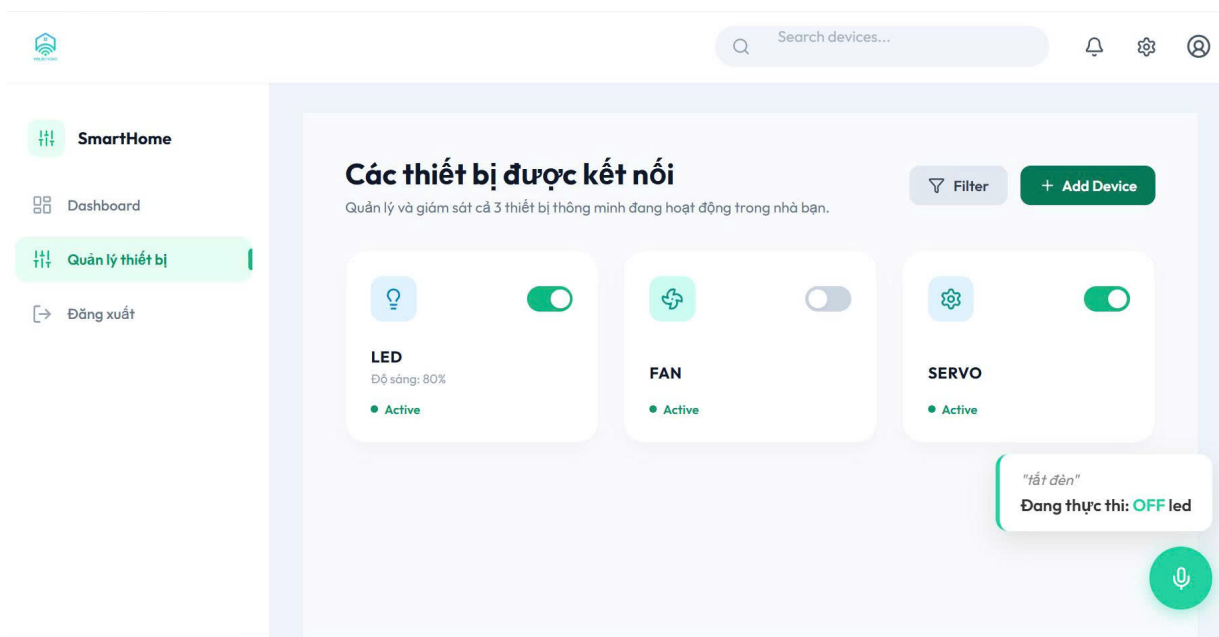
Hình 6: Ảnh chụp giao diện hiển thị hộp cảnh báo vi phạm ngưỡng trên Dashboard

Người dùng có thể chuyển hướng sang trang Quản lý thiết bị (/devices) để kiểm soát và theo dõi trạng thái hiện thời của các thiết bị chấp hành trong nhà bao gồm đèn LED, quạt thông gió và động cơ rèm cửa rục rờ như mô tả trong Figure 7.



Hình 7: Ảnh chụp giao diện trang Quản lý thiết bị và trạng thái bật/tắt thủ công

Cuối cùng, tính năng điều khiển bằng giọng nói tiếng Việt trôi nổi tích hợp ở góc phải chân màn hình Web hiển thị vòng tròn đỏ phát xung lan tỏa khi ghi âm và Toast báo kết quả được ghi nhận tại Figure 8.



Hình 8: Ảnh chụp giao diện nút ghi âm giọng nói tiếng Việt và Toast phản hồi kết quả

6 Tổng kết dự án

6.1 Kết quả đạt được

Sau thời gian tìm hiểu, nghiên cứu và triển khai, nhóm dự án đã hoàn thành các mục tiêu cơ bản được đề ra ban đầu. Cụ thể:

- **Về mặt lý thuyết:** Đã tổng hợp và hệ thống hóa thành công các kiến thức liên quan đến IoT và Trí tuệ nhân tạo, tạo tiền đề vững chắc cho việc thiết kế và xây dựng hệ thống.
- **Về mặt thực tiễn:** Thiết kế và triển khai thành công Nhà thông minh áp dụng Trí tuệ nhân tạo.
- **Về mặt hiệu năng:** Hệ thống hoạt động tương đối ổn định trong môi trường thử nghiệm, giao diện trực quan, logic xử lý rõ ràng và thân thiện với người dùng.

6.2 Hướng phát triển tương lai

Nhằm hoàn thiện sản phẩm và nâng cao giá trị thực tiễn, định hướng phát triển của dự án trong thời gian tới bao gồm:

- **Nâng cấp và mở rộng tính năng:** Nghiên cứu và tích hợp thêm Tiện ích nâng cao để hệ thống thông minh, tự động hóa và đáp ứng sát hơn nhu cầu của người dùng.
- **Tối ưu hóa hệ thống:** Áp dụng các kỹ thuật cấu trúc phần mềm hiện đại để cải thiện tốc độ phản hồi và khả năng mở rộng (scalability).



- **Thử nghiệm và triển khai thực tế:** Đưa sản phẩm vào thử nghiệm với tập người dùng đa dạng hơn để thu thập phản hồi (feedback), từ đó tinh chỉnh luồng nghiệp vụ và hoàn thiện các tiêu chuẩn bảo mật.

Mã nguồn

Mã nguồn toàn bộ dự án YOLOHome được quản lý phiên bản trên nền tảng GitHub tại đường dẫn lưu trữ chính thức sau: Đường dẫn mã nguồn GitHub dự án: [Repository dự án](#).